



INSTITUTE OF SCIENCE TOKYO

MASTER THESIS

Interactive Programming with Algebraic Effect Handlers and Holes

Author:
Zhiqi CHEN

Supervisor:
Youyou CONG

Student Number:
24M38039

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Science
in the*

Programming Research Group
Department of Mathematical and Computing Science

August 17, 2026

Declaration of Authorship

I, Zhiqi CHEN, declare that this thesis titled, “Interactive Programming with Algebraic Effect Handlers and Holes” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date:

INSTITUTE OF SCIENCE TOKYO

Abstract

School of Computing
Department of Mathematical and Computing Science

Master of Science

Interactive Programming with Algebraic Effect Handlers and Holes

by Zhiqi CHEN

Algebraic effect handlers enable uniform expression of diverse computational effects, and are increasingly adopted in production-level programming languages. However, their treatment of continuations as first-class values makes control flow difficult to understand. This difficulty is exacerbated by the fact that programmers can only execute complete programs and obtain feedback from execution results. We solve this problem by integrating holes into effect handlers. Holes enable execution of incomplete programs and inspection of in-scope variables and continuations to support program comprehension and incremental development. The integration is however challenging due to the tension between side effects and the purity required by the semantic safety conditions of holes. In this thesis, we formalize a core calculus with effect handlers and holes, prove the correctness of fill-and-resume in the presence of effects, implement a prototype in the form of a VS Code extension, and conduct a user study to evaluate its usability.

Acknowledgements

First and foremost, I would like to thank my supervisor, Asst. Prof. Youyou Cong, for her continuous guidance and support throughout this thesis. She provided invaluable advice at every stage of the project and encouraged me to participate in the ICFP Student Research Competition, which has been an invaluable experience for both this research and my academic development.

I would like to thank Prof. Hidehiko Masuhara, Asst. Prof. Yudai Tanabe, and the members of the Programming Research Group for their valuable feedback and suggestions during seminars.

I am grateful to Prof. Filip Strömbäck at Linköping University for his keen interest in my work and for his insightful advice on the design of my user study.

I would also like to thank Prof. Kenichi Asai at Ochanomizu University for the inspiring discussions on the semantics of effect handlers, as well as him and the members of his laboratory for their support with the user study.

Finally, I would like to thank my family for their constant love and support.

Contents

Declaration of Authorship	iii
1 Introduction	1
2 Background	5
2.1 Algebraic Effects and Effect Handlers	5
2.1.1 Effects as Typed Operations	5
2.1.2 Handlers and Continuations	5
2.1.3 Deep Handlers	5
2.1.4 A Simple Example	6
2.1.5 Fine-Grain Call-by-Value	6
2.2 Holes and Live Programming	7
2.2.1 Holes	7
2.2.2 The Hazel Programming Environment	7
2.2.3 The Soundness of Fill-and-Resume	8
3 The Challenge of Combining Effect Handlers and Holes	11
3.1 The Core Challenge	11
3.2 Towards a Safe Combination	12
4 Formalization	13
4.1 Core Language Syntax	13
4.2 Small-Step Operational Semantics	15
4.2.1 Reduction rules	15
4.2.2 Why reduction must be allowed under all contexts	17
4.3 Well-Scopedness and Hole Filling	18
4.3.1 Well-Scopedness	18
4.3.2 Hole Filling Function	19
4.4 Properties of $\lambda_{\text{eff}}^?$	21
4.4.1 Commutativity of Hole Filling with Reduction	21
Intuition	21
Proof of Commutativity	22
4.4.2 Confluence	24
Parallel reduction	24
Complete development and the triangle property	25
4.4.3 Fill-and-Resume Soundness	26
4.5 Discussion	27
4.6 Mechanization in Agda	27
5 Implementation	29
5.1 Architecture of the Implementation	29
5.2 Language Design and Implementation	30
5.2.1 Exposing Only Pure Holes	30

5.2.2	Abstract Syntax	30
5.2.3	Elaboration and Well-scopedness	32
5.2.4	Evaluation and Simplification Algorithms	33
5.2.5	Relation to the Formal Definition of $\lambda_{\text{eff}}^?$	36
5.3	Interactive Features and Language Protocol	37
5.3.1	Communication Protocol	37
5.3.2	VS Code Extension Frontend	38
5.3.3	Data Collection for Interactive Features	39
6	Evaluation	41
6.1	Research Questions	41
6.2	Method	41
6.2.1	Participants	42
6.2.2	Study Design	42
6.2.3	Task Design	43
6.2.4	Data Collection & Analysis	44
6.3	Results	45
6.3.1	Task Completion and Hole Usage Overview	45
6.3.2	Holes Support Program Understanding (RQ1)	46
6.3.3	Simplification Supports Continuation Understanding (RQ2)	47
6.3.4	Individual Differences and Experience Level (RQ3)	47
6.3.5	User Feedback and Improvement Suggestions	48
6.4	Discussion	49
7	Related Work	51
8	Conclusion	53
8.1	Conclusion	53
8.2	Future Work	53
	Bibliography	55
A	Source Code for User Study	57

List of Figures

1.1	Screenshot of the <i>Effhole</i> prototype.	3
2.1	Screenshot of the Hazel programming environment. ¹	7
4.1	$\lambda_{\text{eff}}^?$ Core syntax and metavariables	14
4.2	One-step computation reduction $c \longrightarrow c'$	16
4.3	One-step value reduction $v \longrightarrow_v v'$	17
4.4	One-step substitution reduction $\sigma \longrightarrow_s \sigma'$	17
4.5	Substitution of pure and impure holes	18
4.6	Scoping judgment $\Delta; \Gamma \vdash v$, $\Delta; \Gamma \vdash c$ and $\Delta; \Gamma \vdash \sigma : \Gamma'$	19
4.7	Hole filling function $e\llbracket d/i \rrbracket$	20
4.8	Selected parallel reduction rules	25
4.9	Complete development $\mathcal{M}(c)$	26
5.1	Architecture of the prototype implementation of <i>Effhole</i>	29
5.2	<i>Effhole</i> abstract syntax (external and internal)	31
5.3	The elaboration algorithm from external to internal syntax.	32
5.4	Selected cases of the predicate $\text{indet}_E(d)$ for evaluation.	33
5.5	Evaluation contexts E	34
5.6	Redex forms r	34
6.1	Overview of task completion and hole usage across participants.	45
6.2	The hole U2 inserted in Task 3	46

List of Tables

6.1	Overview of study participants and their prior experience with effect handlers.	42
6.2	Overview of study tasks, their target concepts, hole placement, expected need for simplification, and difficulty.	44

Chapter 1

Introduction

In the design of programming languages, a central concern is how to account for computational effects—program behaviors such as mutable state, input/output, exceptions, nondeterminism, and concurrency that go beyond returning a value. Traditionally, languages have either integrated specific effects into their core semantics or required programmers to encapsulate them in ad hoc abstractions. In either case, effects become difficult to define, compose, and reason about in a uniform manner.

Algebraic effects (Plotkin and Power, 2003) and effect handlers (Plotkin and Pretnar, 2013) are a powerful abstraction for modeling and reasoning about computational effects in programming languages. They provide a structured way to define and handle effects, allowing for more modular and composable code. Because of their expressive power, effect handlers have been adopted in real-world programming languages, such as Multicore OCaml (Sivaramakrishnan et al., 2021), and applications, such as probabilistic programming (Bingham et al., 2019).

However, effect handlers can be challenging for programmers to use correctly, especially when it comes to reasoning about the flow of control. Because effectful operations transfer control to handlers and back, the programmer must simultaneously track both the ordinary flow of values and the implicit flow across handler clauses and continuation invocations.

To illustrate this non-sequential control flow, consider the following state handler implemented in a standard effect handler calculus:

$$\begin{aligned} \text{state} = \text{handler} \{ & \text{ret}(x) \mapsto \lambda s. \langle x, s \rangle \\ & ; \text{get}(_, k) \mapsto \lambda s. k \ s \ s \\ & ; \text{put}(v, k) \mapsto \lambda s. k \ () \ v \} \\ (\text{with } \text{state} \text{ handle do } & \{ x \leftarrow \text{get}(); \text{put}(x + 1) \}) \end{aligned} \quad 42$$

Here, $\text{handler}\{\dots\}$ defines a handler with three clauses: one for returning a value $\text{ret}(x)$, two for operations **get**, and **put**; $\text{with } \text{state} \text{ handle } e$ handles the computation e with the handler state ; and $\text{do } \{ x \leftarrow e_1; e_2 \}$ is sequential composition. The variable k in each operation clause is the continuation that captures the “rest of the computation” after the operation.

At first glance, this program appears to be ordinary sequential code: it reads a value, increments it, and writes it back. However, as it is implemented with effect handlers, the control flow is far from sequential. When **get**() is called, execution suspends and jumps to the **get** clause of the handler, which receives the current state s and resumes the continuation k with s as both the returned value and the (unchanged) state. Later, **put**($x + 1$) transfers control to the **put** clause, which resumes k with unit value $()$ and updates the state to $x + 1$. The state s is thus threaded through continuation invocations rather than being an explicit variable in the computation. Consequently, the programmer must mentally reconstruct the sequence

of non-local control transfers in order to understand the program’s behavior, which makes reasoning about it substantially more difficult.

A natural way to help programmers understand such effectful computations is to let them inspect intermediate states of evaluation. Holes, as introduced in the Hazel programming environment (Omar et al., 2019), provide exactly this capability: they allow programmers to leave parts of a program unfinished while still receiving type and evaluation feedback. By inspecting the hole context, programmers can gain insight into the expected structure of subexpressions, which aids both program construction and understanding.

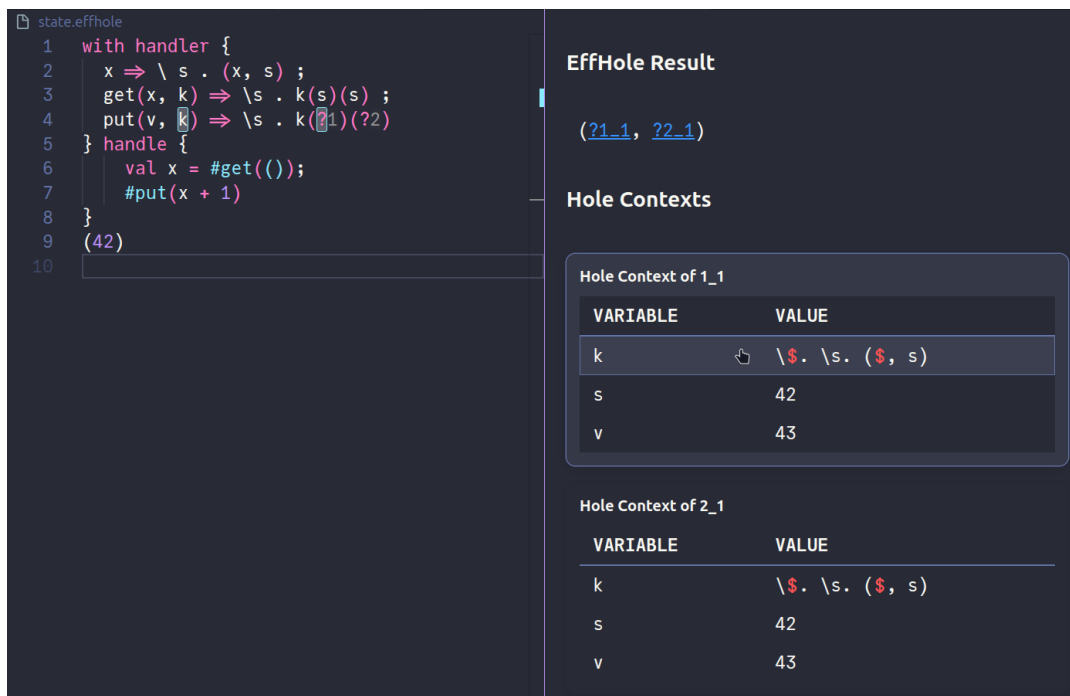
However, the interaction between holes and computational effects has not been explored. Hazel establishes fill-and-resume safety, but only for pure expressions. Because Hazel is pure, a hole can simply be filled with an expression and evaluation resumes from the point of the hole, yielding the same result as evaluating the fully filled program from the beginning. This fill-and-resume safety is central to the use of holes, because it ensures that filling a hole does not introduce unexpected behavior. With computational effects, however, this is no longer the case: when a hole is filled with an effectful expression, resuming evaluation may lead to a different order of operation invocations, which can violate fill-and-resume safety.

In this thesis, we address this gap by investigating the interaction between holes and effect handlers. Concretely, we develop a core calculus, $\lambda_{\text{eff}}^?$, for reasoning formally about this combination, and a prototype programming environment, *Effhole*, that realizes the ideas in practice. We make three contributions:

1. We present $\lambda_{\text{eff}}^?$, a core calculus with a safe semantics for combining holes and effect handlers, ensuring that the fill-and-resume operation is safe for both pure and effectful expressions.
2. We implement *Effhole*, a prototype programming environment that supports effect handlers and holes based on our formalization.
3. We evaluate our work through a user study, demonstrating the effectiveness of our approach in practice.

With these contributions, we aim to provide a foundation for the safe and effective use of holes in the context of effect handlers, enabling programmers to reason about and construct programs with complex control flow more easily. Figure 1.1 shows a screenshot of the *Effhole* prototype. The user may insert holes (written as “?”) into the program, and the hole context displays the current runtime value, helping the programmer reason about the program even in the presence of effect handlers.

The remainder of this thesis is organized as follows. Chapter 2 reviews the background on algebraic effects, effect handlers, and holes. Chapter 3 discusses the core challenge of combining holes with effect handlers and motivates the semantic restrictions needed to preserve fill-and-resume safety. Building on this foundation, Chapter 4 introduces $\lambda_{\text{eff}}^?$, presenting its syntax, operational semantics, and key metatheoretical properties. Chapter 5 describes how this formalization is realized in *Effhole*, a prototype programming environment that supports both effect handlers and holes. Chapter 6 evaluates the approach through a user study that demonstrates its effectiveness in practice. Chapter 7 situates our contributions in relation to existing work on effect handlers, holes, and programming environments. Finally, Chapter 8 summarizes our contributions and discusses directions for future work.

FIGURE 1.1: Screenshot of the *Effhole* prototype.

Chapter 2

Background

2.1 Algebraic Effects and Effect Handlers

Algebraic effects (Plotkin and Power, 2003) and effect handlers (Plotkin and Pretnar, 2013) offer a way to describe computational effects as algebraic operations, providing a generic mechanism for declaring effect operations and for defining handlers that give those operations meaning. This separation between invoking effect operations and defining their meaning via handlers makes programs more modular and composable.

2.1.1 Effects as Typed Operations

An effect is characterized by a collection of operations. For example, a state effect for integer memory cells might provide two operations, **get** and **put**, while an exception effect might provide a single **raise** operation. Each operation has a signature that describes the type of value it expects and the type of value it returns. A program invokes an operation by naming it and supplying the required argument.

2.1.2 Handlers and Continuations

Effect handlers define the meaning of operations. A handler is a collection of clauses: one return clause that describes how to handle a final value, and one clause for each operation the handler interprets. When an operation reaches a handler, the corresponding operation clause is executed.

The most distinctive feature of effect handlers is that each operation clause has access to the *continuation* of the operation call: the “rest of the computation” from the operation call up to the handler. The continuation is typically bound to a variable *k* and can be invoked zero, one, or multiple times. This ability to capture and manipulate continuations is what makes effect handlers expressive enough to encode a wide variety of control effects, including exceptions, state, and nondeterminism (Kammar, Lindley, and Oury, 2013).

For example, a handler for a **chooce** operation that flips a coin might invoke its continuation twice, once with `true` and once with `false`, and collecting both results. A handler for exceptions might ignore the continuation entirely and simply return an error value.

2.1.3 Deep Handlers

Handlers come in two flavors: *deep* and *shallow* (Hillerström and Lindley, 2018). A deep handler automatically re-installs itself around the continuation, so that every operation performed by the continuation is handled by the same handler unless

an inner handler overrides it. A shallow handler, by contrast, processes only the first operation that reaches it; if the continuation performs further operations, those operations escape the handler and must be caught by an outer handler.

In this thesis we focus on deep handlers: the operational semantics of $\lambda_{\text{eff}}^?$ re-installs the handler around the captured continuation whenever an operation is handled.

2.1.4 A Simple Example

To make these ideas concrete, consider a small logging effect with a single operation **log**. The operation **log**(*s*) produces a log message *s*; a handler can interpret these messages by accumulating them in a list, printing them to the console, or discarding them. The following program invokes **log** twice inside a handler that concatenates emitted strings:

$$h = \text{handler } \{x \mapsto \text{ret } x; \mathbf{log}(s, k) \mapsto \text{do } x \leftarrow k () \text{ in ret } (s ++ x)\} \\ \text{with } h \text{ handle do } \{ \mathbf{log}(\text{"Hello "}); \mathbf{log}(\text{"World!"}); \text{ret } "" \}$$

Here $++$ denotes string concatenation. The handler's return clause returns the final value unchanged. The operation clause receives the emitted string *s* and the continuation *k*; it invokes *k* with unit value $()$ to resume the rest of the computation and then appends the emitted string to the result of the continuation. Because the handler is deep, the continuation *k* is wrapped in the same handler, so the second **log** is also interpreted. The program therefore evaluates to `ret "Hello World!"`.

This example illustrates the two key ingredients of effect handlers: operations that stand for effects and handlers that give them meaning by capturing and resuming continuations.

2.1.5 Fine-Grain Call-by-Value

Effect handler calculi are often based on fine-grain call-by-value, a programming-language discipline introduced by Levy, Power, and Thielecke (2003). In fine-grain call-by-value, the syntax is divided into two syntactic categories: *values* and *computations*. Values are pure and effect-free; they include variables, lambda abstractions, and other irreducible data. Computations may perform effects; they include function application, sequencing, operation calls, and effect handling.

This separation makes the flow of effects explicit. A function abstraction $\lambda x. c$ is a value even though its body *c* is a computation. Applying the function produces a computation. Sequencing is written `do  $x \leftarrow c_1$  in  $c_2$` : first evaluate computation c_1 to a value, bind the result to *x*, and then evaluate computation c_2 .

Fine-grain call-by-value is particularly well suited for combining effects with holes because it lets us distinguish between pure holes (which will be filled with values) and impure holes (which may be filled with effectful computations). This distinction is the key to safe hole filling: because a pure hole is guaranteed to be filled with an effect-free value, it can be substituted freely without changing the order of effects. An impure hole carries no such guarantee, so its substitution must be controlled carefully. Our semantics does not, however, forbid evaluation under impure holes; it controls the propagation of effects so that only reductions preserving the order of effects are permitted. In Chapter 4 we exploit this distinction to define a semantics in which hole filling is sound for both kinds of holes. The fine-grain treatment also ensures that effects occur only at explicitly marked computation

positions, which simplifies reasoning about when effects can propagate during evaluation around holes.

2.2 Holes and Live Programming

2.2.1 Holes

A *hole* is a placeholder that a programmer leaves in an incomplete program. It stands for a missing expression that has yet to be supplied. In this thesis we focus on holes in the style of the Hazel programming environment.

A hole can be filled by the programmer with an expression. The key challenge is to give a meaningful semantics to programs that contain holes, so that the programmer receives useful feedback before the program is complete. In particular, the hole must capture the variables that are in scope at its location, so that the programmer can inspect the hole's context and understand what values are available for filling it.

2.2.2 The Hazel Programming Environment

Hazelnut Live, described by Omar et al. (2019), is a live programming environment in which every editor state, including states containing holes, can be dynamically evaluated. Rather than aborting when evaluation reaches a hole, Hazel's semantics *proceeds around* the hole. It records the environment of variables in scope at each hole instance, producing a *hole closure*. The user can inspect the hole closure to see the values of variables available for filling the hole.

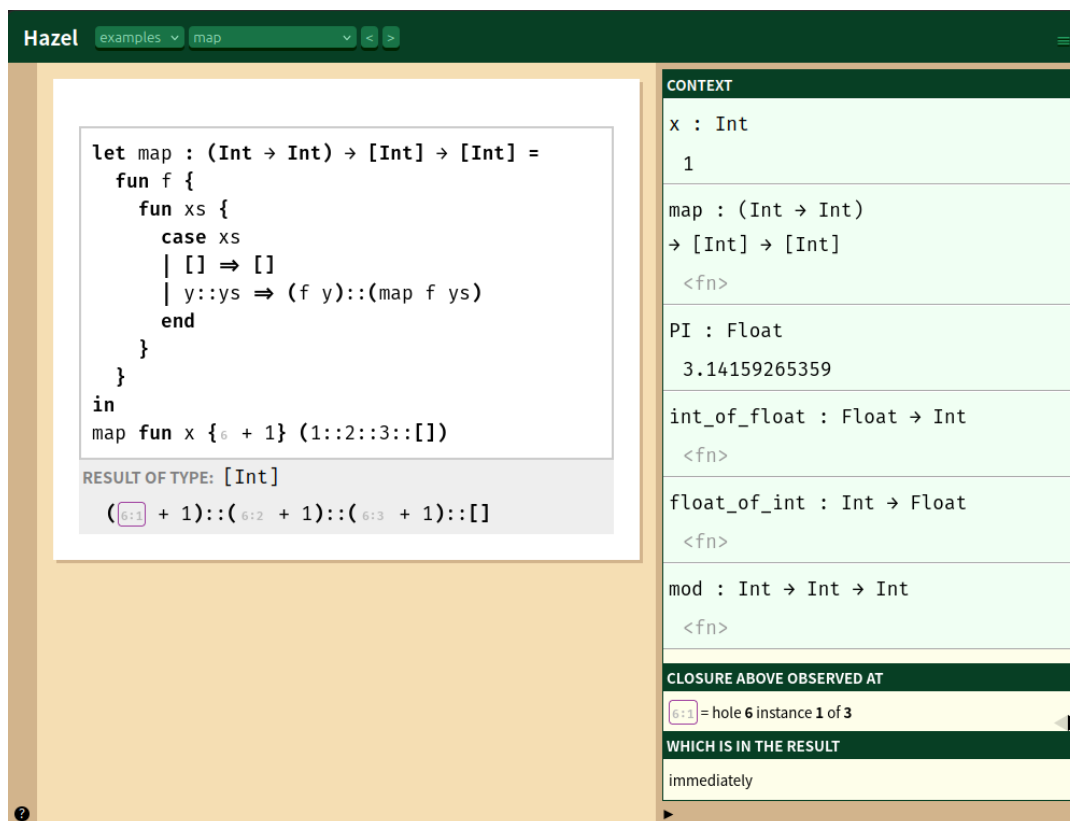


FIGURE 2.1: Screenshot of the Hazel programming environment.¹

Although Hazel’s holes also carry types, our focus in this thesis is on the dynamic behavior of holes and the variables they capture.

Hazel also supports a *fill-and-resume* operation. When a programmer fills a hole with an expression, evaluation can resume from the point where the hole was encountered, rather than restarting the entire program.

2.2.3 The Soundness of Fill-and-Resume

The safety of fill-and-resume operation is a central correctness property: filling a hole and then evaluating should give the same result as evaluating first and then filling the hole.

The fill-and-resume property can be stated informally as follows. Suppose a program c contains a hole u and evaluates to a program c' in which the hole u may still appear, possibly with an updated substitution capturing the values of variables in its context. If we fill u with a well-scoped expression d both before and after evaluation, the two results should be the same.

Omar et al. (2019) formalizes this property in two lemmas: a *commutativity* lemma and a *confluence* lemma.

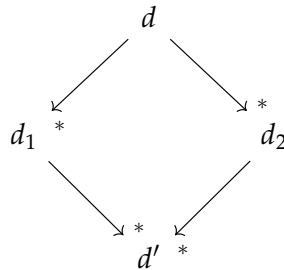
Commutativity

$$\begin{array}{ccc}
 d & \longrightarrow & {}^*d_1 \\
 \vdots \cdot \llbracket d'/u \rrbracket & & \vdots \cdot \llbracket d'/u \rrbracket \\
 \downarrow & & \downarrow \\
 d \llbracket d'/u \rrbracket & \longrightarrow & {}^*d_1 \llbracket d'/u \rrbracket
 \end{array}$$

This commutativity of hole filling and reduction is the key property that ensures that the behavior of a program is not affected before and after filling a hole.

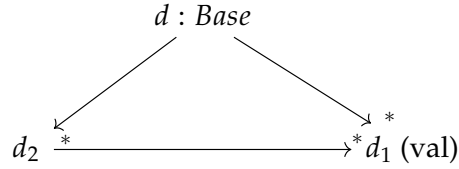
However, commutativity alone is not sufficient to guarantee that the fill-and-resume operation is sound. Suppose we have a semantics that does not satisfy commutativity, we can naively construct a commutativity closure by simply allowing more reduction relations. This closure would satisfy commutativity, but it may still not be the safe semantics we want. Therefore, we also need to ensure that the semantics is confluent.

Confluence Confluence is a core property for a programming language: the order of evaluation should not affect the final result. The standard definition of confluence is that if a program d can reduce to two different programs d_1 and d_2 , then there exists a program d' such that both d_1 and d_2 can reduce to d' :

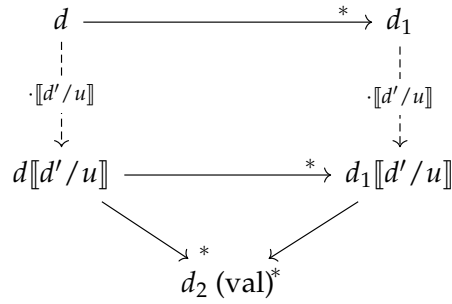


¹The screenshot was taken from an archived build of Hazel available at <https://hazel.org/build-pre-july2024/hazel2>. The current version on hazel.org has since evolved and differs from the system described by Omar et al. (2019).

However, for a practical programming language that does not reduce under binders such as Hazel, this standard definition of confluence is too strong. Therefore, Omar et al. (2019) uses a weaker definition of confluence: for all programs d of base type, if d can reduce to value d_1 , and d can also reduce to d_2 , then d_2 can reduce to d_1 :



Soundness Together, the commutativity and confluence lemmas imply the soundness of the fill-and-resume operation: if a program of base type d evaluates to d_1 and $d \llbracket d' / u \rrbracket$ evaluates to d_2 , then $d_1 \llbracket d' / u \rrbracket$ evaluates to d_2 as well:



Chapter 3

The Challenge of Combining Effect Handlers and Holes

Since Hazel is a pure language, the fill-and-resume soundness theorem established by Omar et al. (2019) is restricted to *pure* expressions. Once effects are involved, the order in which evaluation and hole filling occur can matter.

This limitation is the starting point of our work. We ask whether the fill-and-resume property can be extended to effectful programs, and if so, what semantic restrictions are needed to make it hold.

3.1 The Core Challenge

The interaction between holes and effects raises two competing concerns. On one hand, evaluation should proceed as far as possible even when holes are present, so that the programmer can inspect intermediate states and receive feedback about the program's behavior. On the other hand, the fill-and-resume property must be preserved: filling a hole should commute with evaluation, even when effects are involved.

These two concerns can conflict. If evaluation is allowed to proceed blindly under a hole, then effectful subterms in the continuation may perform effects before the hole is filled. When the hole is later filled with an effectful expression, the combined sequence of effects observed may differ from what would have happened if the hole had been filled first. Thus, a naive combination of holes with effect handlers breaks fill-and-resume property.

Consider the following program, written using the **log** effect from Section 2.1.4. Recall that **log**(*s*) emits a string *s*, and that the deep handler around the program handles each **log** call, resumes the continuation, and concatenates the emitted strings in order:

$$\text{do } \{ x \leftarrow \{?\}^u; \text{log}(\text{"World"}); \text{ret } x \} \quad (1)$$

Here $\{?\}^u$ is an impure hole standing for a missing computation. Suppose the programmer intends to fill the hole with **log**("Hello ").

If we fill the hole first, the program becomes

$$\text{do } \{ x \leftarrow \text{log}(\text{"Hello "}); \text{log}(\text{"World"}); \text{ret } x \}$$

which produces the output "Hello World".

Now suppose instead that evaluation is allowed to proceed around the hole. Then the **log**("World") in the continuation of the hole is reduced first, producing the output "World". When the hole is later filled with **log**("Hello "), that effect is

performed afterward, yielding the output "WorldHello ". The two strategies produce observationally different behavior, so fill-and-resume fails to hold.

This simple example shows that the presence of effects makes the order of hole filling and evaluation observable, which can break the fill-and-resume property. A semantics for holes in an effectful language must therefore control carefully where effects are allowed to propagate in the presence of holes, rather than forbidding evaluation under holes altogether.

3.2 Towards a Safe Combination

The example above suggests a natural design principle: operations should not propagate past an unfilled impure hole to an outer handler. In particular, an operation in the continuation of an impure hole should not be handled by a surrounding handler before the hole itself is filled, because doing so would change the order in which effects are performed.

This does not mean that evaluation should stop entirely around an impure hole. Effects that are *local* to a subterm containing a hole can still be reduced safely. For example, in the program

$$\text{do } \{ x \leftarrow \{?\}_{\sigma'}^u; \text{ with } h \text{ handle } \mathbf{log}(\text{"Local"}) \}$$

Since the $\mathbf{log}(\text{"Local"})$ is handled by the inner handler h , it will not change the behavior of the program no matter how the hole is filled. Therefore, it is safe to reduce this effect even though it occurs in the continuation of an impure hole.

This distinction between local and non-local effects is central to the operational semantics of our formalization. In our formalization, we introduce the distinction of pure and impure holes and use fine-grain call-by-value to define a semantics that precisely controls when effects can propagate, ensuring that fill-and-resume holds for both pure and impure holes. The key difference between the two kinds of holes is substitutability: a pure hole can be filled with a value without changing the order of effects, whereas an impure hole may be filled with an effectful computation, so its substitution must be controlled. The fine-grain operational semantics makes this control explicit while still allowing reductions that do not alter the observable order of effects. The precise formulation of this restriction, together with the corresponding safety theorem, is the subject of Chapter 4.

Chapter 4

Formalization

To formally study the interaction between effect handlers and holes, and reason about its properties, we start by defining a core calculus $\lambda_{\text{eff}}^?$ that captures the essential features. In this chapter, we give the formal definition of the syntax and semantics of $\lambda_{\text{eff}}^?$, as well as the hole filling operation and the well-scopedness judgment needed for this definition. We then establish the key meta-theoretic properties that justify the correctness of $\lambda_{\text{eff}}^?$. Finally, we discuss the implications of our formalization and how it relates to the design of the language and to the prototype implementation. All the formal language definitions and proofs in this chapter are mechanized in the Agda proof assistant.

4.1 Core Language Syntax

$\lambda_{\text{eff}}^?$ is a small, untyped calculus that serves as the core language for our work. Although $\lambda_{\text{eff}}^?$ is minimal, it captures the essential features of effect handlers and holes that are necessary for our theoretical development.

Overview. The syntax of the core language is given in Figure 4.1. The definition follows fine-grain call-by-value (Levy, Power, and Thielecke, 2003), which distinguishes between values (pure terms) and computations (effectful terms) at the syntactic level. The syntax includes standard lambda calculus constructs, as well as constructs for effect handlers and holes.

Effect Handlers. The constructs for effect handlers follow the standard style of Plotkin and Pretnar, 2013, where the operation call $\text{op}(v; y.c)$ represents an operation op is called with argument v , and returns a value which is bound to y in the continuation c . We use $\text{op}(v)$ as a shorthand for $\text{op}(v; y.\text{ret } y)$, which is the common case where the continuation simply returns the value of the operation. This explicit continuation makes the reduction rules local, and simplifies the reasoning about the order of effects. $\text{handler}\{x \mapsto c_r; \text{op}(x, k) \mapsto e\}$ defines a handler with a return clause and an operation clause. The $\text{with } h \text{ handle } c$ construct applies the handler h to the computation c .

Holes. There are two kinds of holes in $\lambda_{\text{eff}}^?$: pure holes, denoted by $?_{\sigma}^i$, and impure holes, denoted by $\{?\}_{\sigma}^i$, where i is the hole identifier and σ is the *hole context*, a partial function from variables to values that records the bindings available at the hole.¹ We assume that the hole identifiers for both pure and impure holes are unique in a program, so that each hole can be uniquely identified by its identifier. The hole

¹The same notion is called a *hole closure* in Hazel (Omar et al., 2019).

Value	$v ::= x \mid \lambda x. c \mid \text{handler}\{x \mapsto c; \text{op}(x, k) \mapsto e\} \mid ?_{\sigma}^i$
Computation	$c ::= \text{ret } v \mid c \mid \text{op}(v; y. c_k) \mid \text{with } v \text{ handle } c$ $\mid \text{do } x \leftarrow c \text{ in } c \mid \{?\}_{\sigma}^i$
Expression	$e, d ::= v \mid c$
	$x, y, k \in \text{Var}$
	$i, u \in \text{HoleId}$
	$\sigma \in \text{Var} \rightarrow \text{Value}$
	$\text{op} \in \text{Operation}$

FIGURE 4.1: $\lambda_{\text{eff}}^?$ Core syntax and metavariables

context captures the free variables of the expression that can be used to fill the hole. During execution, the hole context can be updated to reflect the current values of the free variables at the hole. Note that this calculus is call-by-value, meaning that substitutions only map variables to values and not to computations. We discuss the formal definition of substitution in Section 4.2.

The distinction is based on the observation that the safety of fill-and-resume depends on what kind of expression will fill the hole. A pure hole stands for a missing value: because its eventual fill is guaranteed to be effect-free, it can be substituted freely without changing the order of effects. An impure hole stands for a missing computation: substituting it may introduce or reorder effects, so its fill must be accounted for more carefully. Fine-grain call-by-value makes this distinction explicit in the syntax, and our operational semantics controls the order of effects precisely so that reduction can still proceed around impure holes when it does not change that order. We introduce both kinds of holes so that the programmer can indicate what is known about each missing fragment. In Section 4.4, we show that both kinds of holes enjoy the same fill-and-resume soundness property.

This distinction is not merely a syntactic artifact of fine-grain call-by-value: it reflects a fundamental difference in what we know about the expression that will fill the hole. Our semantics does not, however, forbid all reduction around impure holes. Instead, it controls where effects can propagate so that only reductions preserving the observable order of effects are allowed. For example, the sequencing rule can reduce past a pure hole, as in

$$\text{do } x \leftarrow \text{ret } ?_{\sigma}^i \text{ in ret } x \longrightarrow \text{ret } ?_{\sigma}^i$$

because the hole stands for a value and the right-hand side only needs that value. In contrast,

$$\text{do } x \leftarrow \{?\}_{\sigma}^j \text{ in ret } x$$

cannot reduce by the sequencing rule, because the impure hole does not stand for a value and applying the rule would substitute an unknown computation into the right-hand side. Reductions that do not change the observable order of effects—in particular, reductions handled locally within the continuation of the hole—are still permitted; we discuss this in detail in Section 4.4.1. In Section 4.4, we show that despite this difference in substitutability, both kinds of holes satisfy the same fill-and-resume soundness property. This means that an implementation may offer both kinds of holes to give programmers precise control over where evaluation may proceed, or it may offer only one kind to simplify the surface language, without

sacrificing correctness.

Throughout this chapter, we use v to range over values (pure terms), c to range over computations (effectful terms), x to range over variables, and i, u to range over hole identifiers. We also use e and d to range over expressions, both values and computations, when we want to talk about them uniformly.

In order to focus on the core semantics of holes and effect handlers, the syntax is kept minimal. We omit standard constructs such as booleans, pairs, sums, and recursive functions. These are orthogonal to the core semantics of holes and effect handlers, and can be easily encoded within the lambda calculus (Pierce, 2002).

Similarly, the handlers are only defined for a single operation **op** for simplicity. This simplification does not limit the expressiveness of the language because the standard constructs for multiple operations can be easily encoded using a single operation with tagged arguments. For example, a handler for two operations can be encoded as follows:

$$\begin{aligned} & \text{handler } \{x \mapsto c_r; \mathbf{op}_1(x_1, k) \mapsto c_{op_1}; \mathbf{op}_2(x_2, k) \mapsto c_{op_2}\} \\ &= \text{handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto \text{case } x \text{ of } \{\text{inl } x_1 \mapsto c_{op_1}; \text{inr } x_2 \mapsto c_{op_2}\}\} \end{aligned}$$

4.2 Small-Step Operational Semantics

Next, we define the dynamic behavior of $\lambda_{\text{eff}}^?$ using a small-step operational semantics. In contrast to a standard presentation where reduction is defined only on computations, our semantics defines reduction mutually recursively on values, substitutions, and computations. This design is essential for the meta-theory: because variables may occur in any context and substitution is defined in arbitrary contexts, we need reduction to be available under every syntactic context in order to obtain confluence (Section 4.4.2).

The three one-step reduction relations are:

- $v \longrightarrow_v v'$, reduction of values;
- $\sigma \longrightarrow_s \sigma'$, reduction of substitutions (pointwise, at a single variable); and
- $c \longrightarrow c'$, reduction of computations.

The multi-step reductions $\longrightarrow_v^*$ and $\longrightarrow^*$ are the reflexive-transitive closures of $\longrightarrow_v$ and $\longrightarrow$ respectively, and $\sigma \longrightarrow_s^* \sigma'$ means $\sigma(x) \longrightarrow_v^* \sigma'(x)$ for every variable x .

4.2.1 Reduction rules

The computation reduction rules are given in Figure 4.2. The redexes are standard for fine-grain call-by-value with deep handlers. The structural rules allow reduction under all contexts, including under binders.

Value reduction is given in Figure 4.3. A lambda reduces when its body reduces, a handler reduces when either clause reduces, and a pure hole reduces when its hole context reduces.

Finally, substitution reduction is defined as a single-point update: one variable in the substitution reduces while all other variables remain unchanged.

The substitution operation $e[v/x]$, which replaces all free occurrences of variable x in expression e with value v , follows the standard capture-avoiding substitution. The non-standard cases are for holes and substitutions, which are defined in Figure 4.5.

$$\begin{array}{c}
\text{R-}\beta\text{-APP} \\
\hline
(\lambda x.c)v \longrightarrow c[v/x] \\
\\
\text{R-}\beta\text{-DO} \\
\hline
\text{do } x \leftarrow \text{ret } v \text{ in } c \longrightarrow c[v/x] \\
\\
\text{R-FORWARD-DO} \\
\hline
\text{do } x \leftarrow \mathbf{op}(v; y.c_k) \text{ in } c \longrightarrow \mathbf{op}(v; y.\text{do } x \leftarrow c_k \text{ in } c) \\
\\
\text{R-}\beta\text{-HANDLE-RET} \\
\hline
\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle ret } v \longrightarrow c_r[v/x] \\
\\
\text{R-}\beta\text{-HANDLE-OP} \\
\hline
\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle } \mathbf{op}(v; y.c_k) \\
\longrightarrow c_{op}[v/x, \lambda z.\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle } c_k/k] \\
\\
\text{R-FORWARD-HAND} \\
\hline
\mathbf{op} \neq \mathbf{op}' \\
\hline
\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle } \mathbf{op}'(v; y.c_k) \\
\longrightarrow \mathbf{op}'(v; y.\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle } c_k) \\
\\
\begin{array}{cc}
\text{R-}\xi\text{-RET} & \text{R-}\xi\text{-HOLE-C} \\
\hline
\frac{v \longrightarrow_v v'}{\text{ret } v \longrightarrow \text{ret } v'} & \frac{\sigma \longrightarrow_s \sigma'}{\{?\}_\sigma^i \longrightarrow \{?\}_{\sigma'}^i} \\
\\
\text{R-}\xi\text{-APP1} & \text{R-}\xi\text{-APP2} \\
\hline
\frac{v_1 \longrightarrow_v v'_1}{v_1 v_2 \longrightarrow v'_1 v_2} & \frac{v_2 \longrightarrow_v v'_2}{v_1 v_2 \longrightarrow v_1 v'_2} \\
\\
\text{R-}\xi\text{-DO1} & \text{R-}\xi\text{-DO2} \\
\hline
\frac{c_1 \longrightarrow c'_1}{\text{do } x \leftarrow c_1 \text{ in } c_2 \longrightarrow \text{do } x \leftarrow c'_1 \text{ in } c_2} & \frac{c_2 \longrightarrow c'_2}{\text{do } x \leftarrow c_1 \text{ in } c_2 \longrightarrow \text{do } x \leftarrow c_1 \text{ in } c'_2} \\
\\
\text{R-}\xi\text{-WITH1} & \text{R-}\xi\text{-WITH2} \\
\hline
\frac{h \longrightarrow_v h'}{\text{with } h \text{ handle } c \longrightarrow \text{with } h' \text{ handle } c} & \frac{c \longrightarrow c'}{\text{with } h \text{ handle } c \longrightarrow \text{with } h \text{ handle } c'} \\
\\
\text{R-}\xi\text{-OP1} & \text{R-}\xi\text{-OP2} \\
\hline
\frac{v \longrightarrow_v v'}{\mathbf{op}(v; y.c_k) \longrightarrow \mathbf{op}(v'; y.c_k)} & \frac{c_k \longrightarrow c'_k}{\mathbf{op}(v; y.c_k) \longrightarrow \mathbf{op}(v; y.c'_k)}
\end{array}
\end{array}$$

FIGURE 4.2: One-step computation reduction $c \longrightarrow c'$

$$\begin{array}{c}
\text{R-}\xi\text{-HOLE} \\
\frac{\sigma \longrightarrow_s \sigma'}{?_{\sigma}^i \longrightarrow_v ?_{\sigma'}^i} \\
\\
\text{R-}\xi\text{-ABS} \\
\frac{c \longrightarrow c'}{\lambda x.c \longrightarrow_v \lambda x.c'} \\
\\
\text{R-}\xi\text{-HAND1} \\
\frac{c_r \longrightarrow c'_r}{\text{handler}\{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \longrightarrow_v \text{handler}\{x \mapsto c'_r; \mathbf{op}(x, k) \mapsto c_{op}\}} \\
\\
\text{R-}\xi\text{-HAND2} \\
\frac{c_{op} \longrightarrow c'_{op}}{\text{handler}\{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \longrightarrow_v \text{handler}\{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c'_{op}\}}
\end{array}$$

FIGURE 4.3: One-step value reduction $v \longrightarrow_v v'$

$$\begin{array}{c}
\text{R-}\xi\text{-SUBST} \\
\frac{\exists x. \sigma(x) \longrightarrow_v \sigma'(x) \quad \forall y \neq x. \sigma(y) = \sigma'(y)}{\sigma \longrightarrow_s \sigma'}
\end{array}$$

FIGURE 4.4: One-step substitution reduction $\sigma \longrightarrow_s \sigma'$

Here is an example of how hole contexts work:

$$(\lambda x. ?_{[x/x]}^i) (\lambda y. y) \longrightarrow ?_{[x/x]}^i [\lambda y. y / x] \equiv ?_{[x/x][\lambda y. y / x]}^i \equiv ?_{[\lambda y. y / x]}^i$$

There is a pure hole $?_{[x/x]}^i$ inside a lambda abstraction, and it records the variable x in its hole context. When the lambda abstraction is applied to the value $\lambda y. y$, the hole context of the pure hole is updated to reflect the current value of x at the hole. The resulting term is $?_{[\lambda y. y / x]}^i$, which is a pure hole with an updated hole context that records the current value of x . Suppose we fill the hole with the value x , then the resulting term is $\lambda y. y$, which should be the same as just looking up the value of x in the hole context.

Note that the initial hole context of a hole is always the identity substitution whose domain is the set of free variables at the position of the hole. This ensures that the hole captures all the variables available in its hole context so that the user can refer to them when filling the hole. It is clear that referring to a variable that is not recorded by the hole context is an ill-defined operation. We discuss this further in Section 4.3.1.

4.2.2 Why reduction must be allowed under all contexts

The reduction relation is non-deterministic and allows reduction under all structural contexts. This is a standard approach to establishing the meta-theory of a calculus in a convenient way, which is then used to define a deterministic evaluation strategy and a simplification procedure in Chapter 5.

A stronger motivation for full contextual reduction comes from confluence. To prove confluence, every syntactic construct must be able to reduce inside any context in which it may appear. In particular, variables can appear in arbitrary contexts,

$$\begin{aligned}
?_{\sigma}^i[v/x] &= ?_{\sigma[v/x]}^i \\
\{?\}_{\sigma}^i[v/x] &= \{?\}_{\sigma[v/x]}^i \\
\sigma[v/x] &= (\cdot)[v/x] \circ \sigma
\end{aligned}$$

FIGURE 4.5: Substitution of pure and impure holes

and substitution is defined in arbitrary contexts: $F[x][v/x] = F[v]$. If we want the congruence property $t = t' \Rightarrow F[t] = F[t']$, we must have $t \longrightarrow t' \Rightarrow F[t] \longrightarrow F[t']$ for arbitrary contexts F . The standard semantics, in which reduction is defined only on computations is insufficient: a redex substituted into the right-hand side of a handler or into a hole context becomes “frozen” and no longer reduces, breaking confluence.

For example, consider the redex $(\lambda x. ?_{[x/x]}^i) r$, where r is itself a redex. Reducing the application produces $?_{[r/x]}^i$, and the substituted redex r is now inside a hole context. If substitution itself cannot reduce, the overall term cannot reduce r any further, which leads to diverging reduction sequences that cannot be joined. By giving substitutions their own one-step reduction relation and allowing holes, handlers, and lambdas to reduce structurally, we avoid this problem. In Section 4.4.2, we show that this design choice is crucial for establishing a general confluence property for the calculus instead of just a restricted confluence property as in (Omar et al., 2019).

4.3 Well-Scopedness and Hole Filling

Hole filling is a key operation in $\lambda_{\text{eff}}^?$, and to ensure that it is well-defined, we need to define the notion of well-scopedness for terms. This is crucial because hole filling involves substituting an expression into a hole, and we need to ensure that the free variables of the filling expression are properly accounted for.

4.3.1 Well-Scopedness

To ensure a well-defined hole filling operation, we define well-scopedness for terms. The full set of rules is given in Figure 4.6.

The well-scopedness judgments have the forms $\Delta; \Gamma \vdash v$ and $\Delta; \Gamma \vdash c$, stating that value v and computation c are well-scoped under environment Δ and Γ . Additionally, $\Delta; \Gamma \vdash \sigma : \Gamma'$ states that hole context σ is well-scoped under Δ and Γ , mapping each variable in its domain Γ' to a well-scoped value under Γ .

The judgments use two kinds of environments: a variable environment Γ , and a hole environment Δ . The variable environment $\Gamma \in \mathcal{P}(\text{Var})$ is a set of variables that are currently in scope. Variable binding extends Γ locally: S-ABS and S-BIND extend the environment with the bound variable x for their subterms; S-HANDLER binds x in the return clause and both x and k in the operation clause; S-OP binds y in the continuation.

The hole environment $\Delta \in \text{HoleId} \rightarrow \mathcal{P}(\text{Var})$ is a partial function from hole identifiers to sets of variables, which records the expected free variables of the expression that fills the hole. For instance, rule S-HOLE requires $\Delta(u) \equiv \Gamma'$, meaning pure hole u expects a term with free variables Γ' , and checks that the local hole context σ is well-scoped for that domain.

$\frac{\text{S-SUB} \quad \text{dom}(\sigma) \equiv \Gamma' \quad \forall x \in \Gamma'. \Delta; \Gamma \vdash \sigma(x)}{\Delta; \Gamma \vdash \sigma : \Gamma'}$		$\frac{\text{S-VAR} \quad x \in \Gamma}{\Delta; \Gamma \vdash x}$	$\frac{\text{S-ABS} \quad \Delta; \Gamma, x \vdash c}{\Delta; \Gamma \vdash \lambda x. c}$
$\frac{\text{S-HANDLER} \quad \Delta; \Gamma, x \vdash c_r \quad \Delta; \Gamma, x, k \vdash c_{op}}{\Delta; \Gamma \vdash \text{handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\}}$		$\frac{\text{S-HOLE} \quad \Delta(u) \equiv \Gamma' \quad \Delta; \Gamma \vdash \sigma : \Gamma'}{\Delta; \Gamma \vdash ?_\sigma^u}$	
$\frac{\text{S-RET} \quad \Delta; \Gamma \vdash v}{\Delta; \Gamma \vdash \text{ret } v}$	$\frac{\text{S-APP} \quad \Delta; \Gamma \vdash v_1 \quad \Delta; \Gamma \vdash v_2}{\Delta; \Gamma \vdash v_1 v_2}$	$\frac{\text{S-CHOLE} \quad \Delta(u) \equiv \Gamma' \quad \Delta; \Gamma \vdash \sigma : \Gamma'}{\Delta; \Gamma \vdash \{?\}_\sigma^u}$	
$\frac{\text{S-OP} \quad \Delta; \Gamma \vdash v \quad \Delta; \Gamma, y \vdash c_k}{\Delta; \Gamma \vdash \mathbf{op}(v; y. c_k)}$	$\frac{\text{S-WITH} \quad \Delta; \Gamma \vdash h \quad \Delta; \Gamma \vdash c}{\Delta; \Gamma \vdash \text{with } h \text{ handle } c}$	$\frac{\text{S-BIND} \quad \Delta; \Gamma \vdash c_1 \quad \Delta; \Gamma, x \vdash c_2}{\Delta; \Gamma \vdash \text{do } x \leftarrow c_1 \text{ in } c_2}$	

FIGURE 4.6: Scoping judgment $\Delta; \Gamma \vdash v$, $\Delta; \Gamma \vdash c$ and $\Delta; \Gamma \vdash \sigma : \Gamma'$

The remaining rules are structural, checking subterms in the same environment $\Delta; \Gamma$.

The property that the hole context captures the correct free variables is checked by the well-scopedness judgment and is guaranteed by the elaboration process defined in the next chapter.

4.3.2 Hole Filling Function

With well-scopedness defined, we can now define the hole filling function. The complete definition is given in Figure 4.7. The hole filling function is a mutually recursive function defined on values, computations and substitutions. For readability, we use a uniform notation $e[d/i]$ denoting that the hole with identifier i in expression e is replaced (filled) with expression d . This notation is only a syntactic convenience, and the actual definition distinguishes between pure hole filling $e[v/i]_v$ where i is a pure hole and impure hole filling $e[c/i]_c$ where i is an impure hole. Later in Section 4.4, we show that these two operations satisfy the same soundness property, which justifies the uniform notation.

For example, consider the following term with a pure hole:

$$t = (\lambda x. ?_{[x/x]}^i) (\lambda y. y)$$

If we fill the hole i with the value x , the resulting term is:

$$t[x/i] = (\lambda x. x) (\lambda y. y)$$

As we discussed in Section 4.2, t reduces to $?_{[\lambda y. y/x]}^i$, and filling the hole i with x in this result yields:

$$?_{[\lambda y. y/x]}^i[x/i] = x[\lambda y. y/x] = \lambda y. y$$

As we can see, the result of filling the hole before evaluation is the same as filling the hole after evaluation, which is a key property of hole filling that we prove in

$$\begin{aligned}
x \llbracket d/i \rrbracket &= x \\
(\lambda x. c) \llbracket d/i \rrbracket &= \lambda x. c \llbracket d/i \rrbracket \\
\text{handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \llbracket d/i \rrbracket &= \text{handler } \{x \mapsto c_r \llbracket d/i \rrbracket; \mathbf{op}(x, k) \mapsto c_{op} \llbracket d/i \rrbracket\} \\
(\mathbf{ret } v) \llbracket d/i \rrbracket &= \mathbf{ret } v \llbracket d/i \rrbracket \\
(v_1 v_2) \llbracket d/i \rrbracket &= v_1 \llbracket d/i \rrbracket v_2 \llbracket d/i \rrbracket \\
(\mathbf{do } x \leftarrow c_1 \mathbf{in } c_2) \llbracket d/i \rrbracket &= \mathbf{do } x \leftarrow c_1 \llbracket d/i \rrbracket \mathbf{in } c_2 \llbracket d/i \rrbracket \\
(\mathbf{op}(v; y.c_k)) \llbracket d/i \rrbracket &= \mathbf{op}(v \llbracket d/i \rrbracket; y.c_k \llbracket d/i \rrbracket) \\
(\mathbf{with } h \mathbf{handle } c) \llbracket d/i \rrbracket &= \mathbf{with } h \llbracket d/i \rrbracket \mathbf{handle } c \llbracket d/i \rrbracket
\end{aligned}$$

$$\begin{aligned}
\{?\}_{\sigma}^u \llbracket v/i \rrbracket_v &= \{?\}_{\sigma'}^u \\
?_{\sigma}^u \llbracket v/i \rrbracket_v &= \begin{cases} v[\sigma'] & \text{if } u = i \\ ?_{\sigma'}^u & \text{otherwise} \end{cases} \\
?_{\sigma}^u \llbracket c/i \rrbracket_c &= ?_{\sigma'}^u \\
\{?\}_{\sigma}^u \llbracket c/i \rrbracket_c &= \begin{cases} c[\sigma'] & \text{if } u = i \\ \{?\}_{\sigma'}^u & \text{otherwise} \end{cases} \\
\text{where } \sigma' &= (\cdot) \llbracket d/i \rrbracket \circ \sigma
\end{aligned}$$

FIGURE 4.7: Hole filling function $e \llbracket d/i \rrbracket$

Section 4.4.

Furthermore, it is important to note that if the hole i is filled with a value whose free variables are not recorded by the hole context, the resulting term may be ill-scoped. For example, if we fill the hole i with the value z , whose free variables are not recorded by the hole context, we get:

$$t \llbracket z/i \rrbracket = (\lambda x. z) (\lambda y. y)$$

Therefore, the filling function is defined when e is well-scoped under some environment Δ , such that $\Delta(i) \equiv \Gamma$ for some environment Γ , and d is well-scoped under Γ . In addition, the version of the filling function $e \llbracket d/i \rrbracket$ must be consistent with the type of hole being filled: $e \llbracket v/i \rrbracket_v$ is defined when i is a pure hole and d is a value, and $e \llbracket c/i \rrbracket_c$ is defined when i is an impure hole and d is a computation.

The function proceeds by mutual structural recursion on values, computations, and substitutions. For all constructs other than holes, it descends into the immediate subterms. Since these structural rules are the same for both pure and impure holes, we only give the definition in the form of $e \llbracket d/i \rrbracket$, which is interpreted as $e \llbracket v/i \rrbracket_v$ when d is a value and $e \llbracket c/i \rrbracket_c$ when d is a computation.

The two distinguishing cases are the hole constructs, for pure hole filling:

- For a pure hole $?_{\sigma}^u$: if $u = i$, the hole is replaced by applying the recursively processed substitution σ' to d , written $d[\sigma']$; otherwise the hole is preserved as $?_{\sigma'}^u$.
- For an impure hole $\{?\}_{\sigma}^u$: the hole is always preserved as $\{?\}_{\sigma'}^u$.

The impure hole filling case is just the dual of the pure hole filling case. In both cases, the updated hole context σ' maps each variable x to the recursively filled value $\sigma(x) \llbracket d/i \rrbracket$.

4.4 Properties of $\lambda_{\text{eff}}^?$

In this section, we establish the key meta-theoretic properties of $\lambda_{\text{eff}}^?$. These properties collectively justify that the small-step semantics and hole-filling operation defined in the previous sections are well-behaved and correct. Following the approach of Omar et al., 2019, we prove two central lemmas, commutativity of hole filling with reduction (Theorem 4.1) and confluence of the small-step semantics (Theorem 4.2), and combine them to obtain a general soundness theorem for fill-and-resume (Theorem 4.3).

4.4.1 Commutativity of Hole Filling with Reduction

The first property states that hole filling commutes with reduction: filling a hole (both pure and impure) before evaluating yields the same result (up to multi-step reduction) as evaluating first and then filling the hole.

Theorem 4.1 (Commutativity of Hole Filling). If $\Delta; \Gamma' \vdash d$ and $\Delta[\Gamma'/u]; \Gamma \vdash c$ and $c \longrightarrow^* c'$, then $c \llbracket d/u \rrbracket \longrightarrow^* c' \llbracket d/u \rrbracket$.

Here we use the notation $\Delta[\Gamma'/u]$ to denote the hole environment Δ updated with the mapping $u \mapsto \Gamma'$. The assumption means that the hole u expects a term with free variables Γ' , and we are filling it with a term d that is well-scoped under exactly those variables. More formally, this theorem includes two separate statements, one for pure holes and one for impure holes respectively.

Intuition

At first glance, this property may seem counterintuitive, as one might expect that in the presence of side effects, the order of evaluation could affect the result. It is true that in a language that naively combines holes with side effects, this property would not hold. The following example illustrates this point:

$$\text{do } x \leftarrow \{?\} \text{ in } \text{print}(\text{" World"}) \quad (1)$$

With a naive semantics, if we allow evaluation to proceed under the hole, the `print(" World")` statement would be executed and produce a side effect. However, if we fill the hole first with a computation that produces a side effect, for example, `print("Hello")`, the output would be different. This is because the hole can be understood as a delayed computation, and the order of evaluation can affect the side effects produced. Therefore, it seems that the safe way to combine side effects and hole filling is to disallow evaluation under holes.

However, in $\lambda_{\text{eff}}^?$, we use a more sophisticated semantics that allows evaluation under holes while still ensuring that the order of evaluation does not affect the final result. This is achieved by carefully defining the semantics of hole filling and the reduction rules, ensuring that any side effects produced by the filling expression are properly accounted for and do not interfere with the evaluation of the surrounding computation. The key insight is that the fine-grain call-by-value approach allows us to define a semantics where all the reduction rules are local. In addition, algebraic

effects and handlers provide a structured way to manage side effects, where only the effects that are explicitly handled can be performed.

In our semantics, since `print(" World")` is not handled by any handler in the surrounding computation, it is not a reducible expression in the above example (1). On the other hand, if the expression under the hole is handled by some handler locally, as in the following example:

$$\text{do } x \leftarrow \{?\} \text{ in } \underline{\text{with handler } \{\text{print}(x, k) \mapsto c\} \text{ handle print(" World")}} \quad (2)$$

then, the underlined part is a reducible expression. Here, `with handler` acts as a delimiter for the scope of operation `print`, therefore handling the `print(" World")` call locally does not affect the hole in the surrounding computation. This is the expected behavior of an algebraic effect handler. Finally, the most interesting case is when the whole expression is handled by some handler, as in the following example:

$$\text{with handler } \{\text{print}(x, k) \mapsto c\} \text{ handle do } x \leftarrow \{?\} \text{ in print(" World")} \quad (3)$$

In this case, we must ensure that the `print(" World")` call is not handled by the surrounding handler before the hole is filled. This is also ensured by the semantics of $\lambda_{\text{eff}}^?$, because operations can propagate outwards only if they occur in the position c_1 of `do  $x \leftarrow c_1$  in  $c_2$` , not in c_2 . Therefore, the `print(" World")` call is not handled by the surrounding handler before the hole is filled.

Proof of Commutativity

The proof of commutativity follows the proof sketch given by Omar et al., 2019 and adapts it to $\lambda_{\text{eff}}^?$ with algebraic effect handlers. The key difference is that in $\lambda_{\text{eff}}^?$, we allow reduction under all contexts, including under holes. This makes the original lemma stating that one-step reduction commutes with hole filling no longer hold, because a single step of reduction may require multiple steps after hole filling. Therefore, we need to prove a slightly weaker lemma that every one-step reduction can be mirrored after hole filling up to multi-step reduction. This lemma is still sufficient to prove the commutativity theorem.

The argument is carried out separately for pure hole filling and impure hole filling, but the two cases are perfectly symmetric. We therefore only describe the proof sketch for pure holes and note that the impure case follows the same structure.

Well-scopedness preservation. Before establishing commutativity with reduction, we verify that hole filling is well-defined on well-scoped terms.

Lemma 4.1 (Filling preserves well-scopedness). If $\Delta[\Gamma'/u]; \Gamma \vdash e$ and $\Delta; \Gamma' \vdash v$, then $\Delta; \Gamma \vdash e[v/u]_v$.

Proof. The proof proceeds by mutual structural induction on values, computations, and substitutions. The only interesting case is a pure hole $?_\sigma^u$ being filled: the result is $d[\sigma[d/u]_v]$, which is well-scoped by the induction hypothesis on σ and the assumption on d . All other cases follow immediately from the induction hypotheses. $\square$

Substitution commutes with hole filling. Because hole filling descends into sub-terms and into the substitutions carried by holes, ordinary substitution and hole filling must commute. Let σ be a substitution with domain Γ'' ; we write $\sigma[d/u]_v$ for the substitution mapping each $x \in \Gamma''$ to $\sigma(x)[d/u]_v$.

Lemma 4.2 (Substitution commutes with filling). If $\Delta[\Gamma'/u]; \Gamma'' \vdash e$ and $\Delta[\Gamma'/u]; \Gamma \vdash \sigma : \Gamma''$ and $\Delta; \Gamma' \vdash v$, then $(e[\sigma])\llbracket v/u \rrbracket_v = (e\llbracket v/u \rrbracket_v)[\sigma\llbracket v/u \rrbracket_v]$.

In particular, for single-variable substitution for value v' we have

$$(e[v'/x])\llbracket v/u \rrbracket_v = (e\llbracket v/u \rrbracket_v)[(v'\llbracket v/u \rrbracket_v)/x].$$

Proof. The proof is again by mutual structural induction. Binder cases (λ , sequencing, operation calls, and handlers) require weakening the filling operation under the bound variables; this is handled by the induction hypothesis together with the fact that weakening commutes with filling. $\square$

Substitution preserves reduction. Because reductions can now occur inside substitutions, we need a lemma stating that applying a multi-step-reducing substitution to a term produces a multi-step reduction of the resulting term.

Lemma 4.3 (Substitution preserves multi-step reduction). If $\sigma \longrightarrow_s^* \sigma'$, then $e[\sigma] \longrightarrow_v^* e[\sigma']$ for values and $c[\sigma] \longrightarrow^* c[\sigma']$ for computations.

Proof. By mutual structural induction on e and c . The key observation is that a single substitution step $\sigma \longrightarrow_s \sigma'$ changes only one variable x , so it yields a single reduction at every occurrence of x in the term. The hole cases use the fact that a reduction of a hole context induces a reduction of the corresponding pure or impure hole. The binder cases use the fact that lifting a pointwise substitution reduction preserves the pointwise property. $\square$

Weak commutativity. The core of the commutativity proof is no longer a one-step commutativity lemma, because filling after a single substitution step may require more than one step in the filled term. For example, for the pure hole $d = ?_\sigma^u$ case, if $\sigma(x) \longrightarrow_v \sigma'(x)$ and the filled computation d contains two free occurrences of x , then $d\llbracket \sigma(x)/u \rrbracket_v$ reduces in two steps to $d\llbracket \sigma'(x)/u \rrbracket_v$. We therefore state a slightly weaker, but still sufficient, lemma.

Lemma 4.4 (Weak commutativity). If $\Delta; \Gamma' \vdash v$ and $\Delta[\Gamma'/u]; \Gamma \vdash c$ and $c \longrightarrow c'$, then $c\llbracket v/u \rrbracket_v \longrightarrow^* c'\llbracket v/u \rrbracket_v$.

Proof. The proof proceeds by case analysis on the reduction derivation:

- The structural ξ rules follow from the induction hypothesis and the congruence lemmas for multi-step reduction. For example, from $c_1 \longrightarrow c'_1$ we obtain $c_1\llbracket v/u \rrbracket_v \longrightarrow^* c'_1\llbracket v/u \rrbracket_v$, and therefore do $x \leftarrow c_1\llbracket v/u \rrbracket_v$ in $c_2\llbracket v/u \rrbracket_v \longrightarrow^*$ do $x \leftarrow c'_1\llbracket v/u \rrbracket_v$ in $c_2\llbracket v/u \rrbracket_v$.
- The β rules for function application and sequencing use the single-variable substitution-commutativity lemma. For instance, the sequencing rule do $x \leftarrow \text{ret } v' \text{ in } c \longrightarrow c[v'/x]$ becomes, after filling,

$$\text{do } x \leftarrow \text{ret}(v'\llbracket v/u \rrbracket_v) \text{ in } c\llbracket v/u \rrbracket_v \longrightarrow^* (c\llbracket v/u \rrbracket_v)[(v'\llbracket v/u \rrbracket_v)/x],$$

which is exactly an instance of the same sequencing rule on the filled terms.

- The handler return rule is similar.

- The most involved case is the handler operation rule $R\text{-}\beta\text{-OP-EQ}$. When a handled operation matches the handler, the continuation is wrapped in a recursive copy of the handler. Filling must distribute through this recursive handler construction; this is the only case that requires a separate auxiliary argument, mirroring the construction of the continuation value in the operational semantics.
- The $R\text{-}\beta\text{-OP-NEQ}$ case follows from the induction hypothesis on the continuation.
- The new value-level structural rules (lambda, handler, pure hole) are handled by the corresponding induction hypotheses on subterms, using the fact that value-level multi-step reduction lifts through each value constructor. $\square$

Multi-step commutativity.

Proof. Finally, the main theorem follows by induction on the multi-step reduction $c \longrightarrow^* c'$. If the sequence is empty, the result is trivial. For a sequence $c \longrightarrow c_1 \longrightarrow^* c'$, Lemma 4.4 gives $c[d/u] \longrightarrow^* c_1[d/u]$, and the induction hypothesis yields $c_1[d/u] \longrightarrow^* c'[d/u]$. Composing these gives the desired $c[d/u] \longrightarrow^* c'[d/u]$. This completes the proof of Theorem 4.1. $\square$

4.4.2 Confluence

Having established that hole filling commutes with reduction, we now turn to the second fundamental property of the semantics: confluence. Confluence guarantees that, no matter which reduction sequence is chosen, any two reduction sequences from the same term can always be joined by further reduction. This property is important for reasoning about program equivalence and for justifying the soundness of the fill-and-resume operation discussed in Section 4.4.3.

Theorem 4.2 (Confluence). If $c \longrightarrow^* c_1$ and $c \longrightarrow^* c_2$, then there exists c' such that $c_1 \longrightarrow^* c'$ and $c_2 \longrightarrow^* c'$.

We prove confluence using the standard technique of parallel reduction together with a complete development function, following the method of Takahashi, 1995. The overall strategy is as follows. We define a parallel reduction relation $\Rightarrow$ that permits any number of redexes in a term to reduce simultaneously. We then define a complete development function $\mathcal{M}$ such that $c \Rightarrow \mathcal{M}(c)$ and, whenever $c \Rightarrow c'$, also $c' \Rightarrow \mathcal{M}(c)$. The latter property is the triangle property; it immediately entails the diamond property for parallel reduction, and from that we obtain confluence of the small-step relation because every single-step reduction is a parallel reduction and every parallel reduction can be simulated by multi-step reduction.

Parallel reduction

The parallel reduction relation $c \Rightarrow c'$ is defined mutually recursively with value-level parallel reduction $v \Rightarrow_v v'$ and pointwise value-level parallel reduction on substitutions $\sigma \Rightarrow_s \sigma'$. Intuitively, $c \Rightarrow c'$ allows all redexes at the top level and inside subterms to reduce in one step, while also keeping subterms unchanged. The key clauses are given in Figure 4.8; the remaining clauses are congruence rules analogous to the structural rules of the small-step semantics.

$$\begin{array}{c}
\begin{array}{c} \text{P-RET} \\ \frac{v \Rightarrow_v v'}{\text{ret } v \Rightarrow \text{ret } v'} \end{array} \qquad \begin{array}{c} \text{P-HOLE-V} \\ \frac{\sigma \Rightarrow_s \sigma'}{?_{\sigma}^i \Rightarrow_v ?_{\sigma'}^i} \end{array} \qquad \begin{array}{c} \text{P-HOLE-C} \\ \frac{\sigma \Rightarrow_s \sigma'}{\{?\}_{\sigma}^i \Rightarrow \{?\}_{\sigma'}^i} \end{array} \qquad \begin{array}{c} \text{P-ABS} \\ \frac{c \Rightarrow c'}{\lambda x.c \Rightarrow_v \lambda x.c'} \end{array} \\[10pt]
\begin{array}{c} \text{P-HANDLER} \\ \frac{c_r \Rightarrow c'_r \quad c_{op} \Rightarrow c'_{op}}{\text{handler}\{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \Rightarrow_v \text{handler}\{x \mapsto c'_r; \mathbf{op}(x, k) \mapsto c'_{op}\}} \end{array} \\[10pt]
\begin{array}{c} \text{P-}\beta\text{-APP} \\ \frac{c \Rightarrow c' \quad v \Rightarrow_v v'}{(\lambda x.c) v \Rightarrow c'[v'/x]} \end{array} \qquad \begin{array}{c} \text{P-FORWARD-DO} \\ \frac{v \Rightarrow_v v' \quad c \Rightarrow c'}{\text{do } x \leftarrow \text{ret } v \text{ in } c \Rightarrow c'[v'/x]} \end{array} \\[10pt]
\begin{array}{c} \text{P-OP-DO} \\ \frac{v \Rightarrow_v v' \quad c_k \Rightarrow c'_k \quad c \Rightarrow c'}{\text{do } x \leftarrow \mathbf{op}(v; y.c_k) \text{ in } c \Rightarrow \mathbf{op}(v'; y.\text{do } x \leftarrow c'_k \text{ in } c')} \end{array} \\[10pt]
\begin{array}{c} \text{P-}\beta\text{-HANDLE-RET} \\ \frac{c_r \Rightarrow c'_r \quad v \Rightarrow_v v'}{\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle ret } v \Rightarrow c'_r[v'/x]} \end{array} \\[10pt]
\begin{array}{c} \text{P-}\beta\text{-HANDLE-OP} \\ \frac{c_r \Rightarrow c'_r \quad c_{op} \Rightarrow c'_{op} \quad v \Rightarrow_v v' \quad c_k \Rightarrow c'_k}{\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle } \mathbf{op}(v; y.c_k) \Rightarrow c'_{op}[v'/x, \lambda z.\text{with handler } \{x \mapsto c'_r; \mathbf{op}(x, k) \mapsto c'_{op}\} \text{ handle } c'_k/k]} \end{array} \\[10pt]
\begin{array}{c} \text{P-FORWARD-HAND} \\ \frac{c_r \Rightarrow c'_r \quad c_{op} \Rightarrow c'_{op} \quad v \Rightarrow_v v' \quad c_k \Rightarrow c'_k \quad \mathbf{op} \neq \mathbf{op}'}{\text{with handler } \{x \mapsto c_r; \mathbf{op}(x, k) \mapsto c_{op}\} \text{ handle } \mathbf{op}'(v; y.c_k) \Rightarrow \mathbf{op}'(v'; y.\text{with handler } \{x \mapsto c'_r; \mathbf{op}(x, k) \mapsto c'_{op}\} \text{ handle } c'_k)} \end{array}
\end{array}$$

FIGURE 4.8: Selected parallel reduction rules

The remaining congruence rules mirror the structural rules of the small-step semantics and are omitted for brevity. Parallel reduction is reflexive, transitive (when taking its reflexive-transitive closure), and contains the ordinary one-step reduction relation.

Complete development and the triangle property

The complete development $\mathcal{M}(c)$ is defined by structural recursion on c (and likewise on values and substitutions); it reduces every redex at the current level after recursively developing the subterms. The definition is shown in Figure 4.9; the value cases are analogous. We extend $\mathcal{M}$ to substitutions pointwise: $\mathcal{M}(\sigma)(x) = \mathcal{M}(\sigma(x))$.

Lemma 4.5 (Triangle property). If $c \Rightarrow c'$, then $c' \Rightarrow \mathcal{M}(c)$.

Proof. By mutual structural induction on the parallel reduction derivation. The interesting cases are the redex rules. For example, for the β rule, if $(\lambda x.c) v \Rightarrow c'[v'/x]$ with $c \Rightarrow c'$ and $v \Rightarrow_v v'$, then by the induction hypotheses $c' \Rightarrow \mathcal{M}(c)$ and $v' \Rightarrow_v$

$$\begin{aligned}
\mathcal{M}(\text{ret } v) &= \text{ret } \mathcal{M}(v) \\
\mathcal{M}((\lambda x.c) v) &= \mathcal{M}(c)[\mathcal{M}(v)/x] \\
\mathcal{M}(\text{do } x \leftarrow \text{ret } v \text{ in } c) &= \mathcal{M}(c)[\mathcal{M}(v)/x] \\
\mathcal{M}(\text{do } x \leftarrow \text{op}(v; y.c_k) \text{ in } c) &= \text{op}(\mathcal{M}(v); y.\text{do } x \leftarrow \mathcal{M}(c_k) \text{ in } \mathcal{M}(c)) \\
\mathcal{M}(\text{with } h \text{ handle ret } v) &= \mathcal{M}(c_r)[\mathcal{M}(v)/x] \\
\mathcal{M}(\text{with } h \text{ handle op}(v; y.c_k)) &= \begin{cases} c'_{op}[v'/x, \lambda z. w'/k] & \text{if } \text{op} = \text{op}' \\ \text{op}(v'; y.w') & \text{if } \text{op} \neq \text{op}' \end{cases} \\
\mathcal{M}(\{?\}_{\sigma}^i) &= \{?\}_{\mathcal{M}(\sigma)}^i \\
\mathcal{M}(v_1 v_2) &= \mathcal{M}(v_1) \mathcal{M}(v_2) \\
\mathcal{M}(\text{do } x \leftarrow c_1 \text{ in } c_2) &= \text{do } x \leftarrow \mathcal{M}(c_1) \text{ in } \mathcal{M}(c_2) \\
\mathcal{M}(\text{with } h \text{ handle } c) &= \text{with } \mathcal{M}(h) \text{ handle } \mathcal{M}(c) \\
\mathcal{M}(\text{op}(v; y.c_k)) &= \text{op}(\mathcal{M}(v); y.\mathcal{M}(c_k))
\end{aligned}$$

where $h = \text{handler } \{x \mapsto c_r; \text{op}(x, k) \mapsto c_{op}\}$,
 $h' = \text{handler } \{x \mapsto \mathcal{M}(c_r); \text{op}(x, k) \mapsto \mathcal{M}(c_{op})\}$, and $w' = \text{with } h' \text{ handle } \mathcal{M}(c'_k)$

FIGURE 4.9: Complete development $\mathcal{M}(c)$

$\mathcal{M}(v)$. The substitution lemma for parallel reduction gives $c'[v'/x] \Rightarrow \mathcal{M}(c)[\mathcal{M}(v)/x] = \mathcal{M}((\lambda x.c) v)$. The handler operation case is the most involved, requiring the auxiliary lemmas that parallel reduction commutes with renaming and substitution in order to match the lifted handler clauses in the complete development. $\square$

From Lemma 4.5 we immediately obtain the diamond property for parallel reduction: if $c \Rightarrow c_1$ and $c \Rightarrow c_2$, then both $c_1 \Rightarrow \mathcal{M}(c)$ and $c_2 \Rightarrow \mathcal{M}(c)$. A standard strip lemma then lifts the diamond property from single parallel steps to the reflexive-transitive closure, giving confluence of parallel reduction. Since every single-step reduction is a parallel reduction and every parallel reduction can be simulated by a multi-step reduction, Theorem 4.2 follows.

4.4.3 Fill-and-Resume Soundness

A direct corollary of commutativity (Theorem 4.1) and confluence (Theorem 4.2) is a general soundness theorem for fill-and-resume. In the appendix of the Hazel paper (Omar et al., 2019), the authors prove only a restricted form of soundness because their semantics does not support reduction under binders: for closed terms of base type, if a term reduces to a value and the filled term also reduces to a value, then those values are equal. Because $\lambda_{\text{eff}}^?$ is confluent in the general case, we can state a more powerful theorem.

Theorem 4.3 (Fill-and-resume soundness). If $\Delta[\Gamma'/u]; \Gamma \vdash d$ and $\Delta; \Gamma' \vdash d'$ and $d \longrightarrow^* d_1$ then there exists d_2 such that $d[d'/u] \longrightarrow^* d_2$ and $d_1[d'/u] \longrightarrow^* d_2$.

In other words, evaluating first and then filling produces a term that can be joined with the term obtained by filling first and then evaluating.

Proof. By Theorem 4.1, $d \longrightarrow^* d_1$ implies $d[d'/u] \longrightarrow^* d_1[d'/u]$. We therefore have two reduction sequences starting from $d[d'/u]$: one to itself by reflexivity and one to $d_1[d'/u]$. Confluence (Theorem 4.2) gives the common reduct d_2 . $\square$

When d_2 is a normal form, Theorem 4.3 specializes to the familiar statement: if filling first leads to a normal form, then evaluating first and then filling leads to the same normal form (up to reduction to it). Our general statement, however, also covers terms that are not closed and not of base type, and terms that may reduce to non-normal forms.

4.5 Discussion

Relation to the standard semantics of algebraic effects handlers. Since our motivation is to help programmers understand and reason about programs with effect handlers, we ensure that the semantics of $\lambda_{\text{eff}}^?$ is consistent with the standard semantics of algebraic effects handlers. The semantics of $\lambda_{\text{eff}}^?$ defined in Section 4.2 is a natural extension of the standard semantics of algebraic effects handlers (Plotkin and Power, 2003; Plotkin and Pretnar, 2013). For the programs that do not contain holes, the semantics of $\lambda_{\text{eff}}^?$ is equivalent to the standard semantics. The main design choice in $\lambda_{\text{eff}}^?$ is the interaction between holes and effectful computations. We allow evaluation to proceed under holes and ensure the fill-and-resume property, which is an intuitive design choice for a language with holes.

Implications for language design. The soundness theorem applies uniformly to both pure and impure holes, even though the two kinds of holes behave differently. This uniformity is one of the main contributions of the formalization: it shows that the distinction between holes that will be filled with values and holes that may be filled with effectful computations does not force a particular implementation strategy. Fine-grain call-by-value makes this distinction syntactically explicit, but the underlying observation that the fill-and-resume property holds as long as the order of effects is preserved after hole filling, is independent of any specific formalism. A surface language may offer both pure and impure holes, giving programmers fine-grained control over where evaluation may proceed; alternatively, it may offer only pure holes, as in the prototype described in Chapter 5, trading expressiveness for a simpler user model. In either case, the fill-and-resume property remains sound.

4.6 Mechanization in Agda

To ensure the correctness of our proofs, we mechanize the definitions and proofs in Agda. The Agda code is approximately 3500 lines and can be found at <https://github.com/prg-titech/effhole-form>. The mechanization includes the definitions of the syntax, semantics, the proof of commutativity of hole filling, and the proof of confluence.

The mechanization follows a similar structure to the presentation in this chapter, except for adapting De Bruijn indices for variable binding and substitution. This technique is commonly used in mechanized proofs to avoid issues with variable capture and alpha-equivalence and is well-known to be equivalent to the named variable approach presented in this chapter (Pierce, 2002). The mechanization also includes additional lemmas and properties that are necessary for the proofs but are omitted in the presentation for brevity.

In addition, the definitions of well-scopedness use a slightly different approach in the mechanization. Instead of using a uniform hole environment Δ for both pure and impure holes, we use two separate environments Δ_v and Δ_c for pure and impure holes respectively. This approach simplifies the mechanization and avoids the need

for additional lemmas to relate the two environments. However, the two approaches are equivalent and the choice of using a uniform environment in the presentation is for clarity and simplicity.

The entire mechanization is verified using Agda v2.8.0 with the `--safe` flag for checking soundness of the proofs. The development relies on no axioms except for function extensionality, whose definition is imported from the Agda standard library. Function extensionality states that two functions are equal if they produce the same output for all inputs: $\forall fg. (\forall x. f\ x \equiv g\ x) \rightarrow f \equiv g$. This axiom is well-known to be consistent but unprovable in standard intensional type theory (Garner, 2009), and does not break the soundness of the mechanization. The use of function extensionality is standard and simplifies the proofs of equality of substitutions which are represented as functions.

Chapter 5

Implementation

Based on the $\lambda_{\text{eff}}^?$ calculus formalized in Chapter 4, we implement *Effhole*, a prototype programming language and interactive environment built on $\lambda_{\text{eff}}^?$. We present the implementation architecture, the design of the surface and internal languages, the evaluation and simplification algorithms, and their correspondence to the formal semantics of $\lambda_{\text{eff}}^?$. We then describe the interactive programming environment, the communication protocol between the editor frontend and the language server, and the data collected to support future research on programming with holes and effects.

5.1 Architecture of the Implementation

The prototype implementation of *Effhole* includes a programming language and an interactive programming environment. The programming language is implemented as a command-line tool that can be used as a normal interpreter for *Effhole* programs, or as a backend language server working with an interactive programming environment through the HTTP protocol. Therefore, the implementation of a frontend for the interactive programming environment is separated from the implementation of the programming language. The architecture of the implementation is shown in Figure 5.1.

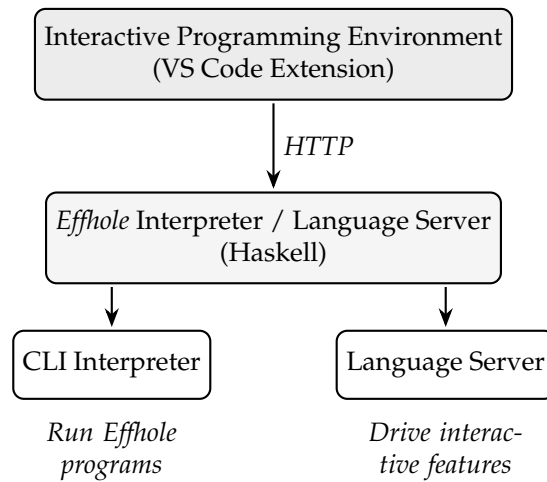


FIGURE 5.1: Architecture of the prototype implementation of *Effhole*.

The language is implemented in Haskell, which is well-suited for implementing interpreters and manipulating abstract syntax with binders. The interactive environment is implemented as a VS Code extension.

The advantage of combining the interpreter and the language server in one backend is that it keeps the editor frontend thin: it is only responsible for rendering results and forwarding user actions. Communication over HTTP using JSON keeps the protocol simple and editor-agnostic; a different frontend—for example, a web interface or another editor—can reuse the same backend with minimal changes. A typical interaction proceeds as follows: the user invokes a command in VS Code, the extension sends the program text and action to the backend, the backend parses, elaborates, and evaluates the program, and the backend returns a JSON response that the extension renders in the output panel.

5.2 Language Design and Implementation

In $\lambda_{\text{eff}}^?$, we include minimal features to keep the calculus simple. In *Effhole*, to make the language more practical and usable, we extend the core calculus with additional features. This section describes the design and implementation of the programming language, including the syntax, semantics, and evaluation strategies.

5.2.1 Exposing Only Pure Holes

In $\lambda_{\text{eff}}^?$, pure and impure holes differ in what may fill them: a pure hole expects a value, while an impure hole expects a computation. We design *Effhole* to expose only pure holes, for the following reasons:

No built-in fill-and-resume command. Chapter 4 defines hole filling as a meta-operation on terms, but *Effhole* does not provide it as a run-time editor command. Instead, the programmer edits the source directly, replacing a hole with a program fragment, and then re-runs the program. The fill-and-resume soundness theorem guarantees that this manual edit-then-run workflow is semantically equivalent to the formal operation of filling the hole in a residual program: evaluating the edited program produces the same result as evaluating to a residual and then filling its hole.

The purity of the fill cannot be enforced. Because filling happens through ordinary source editing, the tool has no reliable way to check whether the expression written in place of a hole is a value or a computation. The current prototype lacks a type and effect system that would reject a pure hole being filled with an effectful expression. If the programmer writes an impure computation where a pure hole is expected, the result may be surprising, because the evaluator and simplifier assume the fill is effect-free.

Given this limitation, in our prototype we decided to expose only pure holes. Pure holes yield simpler evaluation results and impose less cognitive load, because the system can treat every hole as standing for a value. The risk of filling a pure hole with an impure expression is therefore left to the programmer in the current prototype.

5.2.2 Abstract Syntax

The syntax of $\lambda_{\text{eff}}^?$ defined in Section 4.1 serves as a core for *Effhole*. However, its raw constructs are difficult for programmers to write directly. Therefore, *Effhole* has an

external syntax for source programs and an internal syntax used for execution and display.

$$\begin{aligned}
 \text{External } e &::= x \mid b \mid n \mid \text{str} \mid () \mid e_1 \oplus e_2 \mid [] \mid e_1 :: e_2 \\
 &\mid \lambda x_1 \cdots x_n. e \mid (e_1, e_2) \mid \text{inl } e \mid \text{inr } e \\
 &\mid \text{case } e \text{ of } \{\text{inl } x \mapsto e_1; \text{inr } y \mapsto e_2\} \\
 &\mid \text{uncons } e_1 \text{ of } \{[] \mapsto e_2; (x, xs) \mapsto e_3\} \\
 &\mid \text{unpair } e_1 \text{ of } (x, y) \mapsto e_2 \\
 &\mid \text{handler}\{x \mapsto e; \text{op}_i(x_i, k_i) \mapsto e_i\} \mid ?^i \\
 &\mid e_1 e_2 \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mid \text{op}(e) \mid \text{with } e_1 \text{ handle } e_2 \mid \text{do } \{\bar{s}; e\} \\
 \text{DoStmt } s &::= x \leftarrow e \mid e \mid \text{letrec } f \ x_1 \cdots x_n = e_1 \text{ in } e_2 \\
 \text{Internal } d &::= x \mid b \mid n \mid \text{str} \mid () \mid d_1 \oplus d_2 \mid [] \mid d_1 :: d_2 \\
 &\mid \lambda x. d \mid (d_1, d_2) \mid \text{inl } d \mid \text{inr } d \\
 &\mid \text{case } d \text{ of } \{\text{inl } x \mapsto d_1; \text{inr } y \mapsto d_2\} \\
 &\mid \text{uncons } d_1 \text{ of } \{[] \mapsto d_2; (x, xs) \mapsto d_3\} \\
 &\mid \text{unpair } d_1 \text{ of } (x, y) \mapsto d_2 \\
 &\mid \text{handler}\{x \mapsto d; \text{op}_i(x_i, k_i) \mapsto d_i\} \mid ?_\sigma^i \\
 &\mid d_1 d_2 \mid \text{if } d_1 \text{ then } d_2 \text{ else } d_3 \mid \text{op}(d) \mid \text{with } d_1 \text{ handle } d_2 \\
 &\mid \text{do } x \leftarrow d_1 \text{ in } d_2 \mid \text{letrec } f \ x = d_1 \text{ in } d_2
 \end{aligned}$$

$$\begin{aligned}
 &b \in \text{Bool}, \quad n \in \text{Int}, \quad \text{str} \in \text{String} \\
 &x, y, k, f, xs \in \text{Id}, \quad i \in \text{HoleId}, \quad \sigma \in \text{Id} \rightarrow \text{Internal Expr}
 \end{aligned}$$

FIGURE 5.2: *Effhole* abstract syntax (external and internal)

During elaboration, external constructs are desugared into the simpler internal forms shown in Figure 5.2. The internal syntax is close to the external one—both share variables, literals, applications, handlers, and most constructors—but it uses single-parameter abstractions, a binary sequencing construct, and hole forms annotated with substitutions. Keeping the two levels close makes it easier for the user to understand the output result.

There are several main differences between the *Effhole* syntax and the core calculus of $\lambda_{\text{eff}}^?$ defined in Section 4.1. First, *Effhole* is not fine-grain call-by-value, and it does not distinguish between values and computations. Second, there is only one kind of holes in *Effhole*, which behave the same as pure holes in $\lambda_{\text{eff}}^?$. The rationale for exposing only pure holes is given in Section 5.2.1.

Compared to $\lambda_{\text{eff}}^?$, the external syntax of *Effhole* includes several convenient constructs. It provides primitive support for booleans, integers, strings, and unit, as well as compound data types such as lists and pairs.

Functions can be written with multiple parameters, $\lambda x_1 \cdots x_n. e$, and recursive functions can be defined with `letrec`. The `do` notation offers a more readable syntax for sequencing effectful computations, and pattern-matching constructs such as `uncons`, `unpair`, and `case` eliminate the need for explicit projection functions.

Operation calls are written as `op(e)`, and handlers use a conventional handler syntax with named return and operation clauses.

These external constructs are desugared into the internal syntax during elaboration. For example, a multi-parameter abstraction $\lambda x_1 \cdots x_n. e$ becomes a nest of single-parameter abstractions, and a `do` block is expanded into a sequence of

do $x \leftarrow d_1$ in d_2 and `letrec` bindings. The internal syntax therefore closely follows the core calculus, but represents bound variables using Haskell’s `Bound` library (Kmetz, 2023): subterms under binders are stored as *Scope* values. In Figure 5.2, we write $\lambda x. d$ and $\lambda xk. d$ for such scoped subterms, where the indicated variables are bound in d . The most important difference between the two syntactic levels is the treatment of holes: external holes are bare identifiers $?^i$, whereas internal holes carry a substitution recording the values of variables available in the hole’s context, written $?^i_{\sigma}$. Section 5.2.3 explains how elaboration constructs this substitution.

5.2.3 Elaboration and Well-scopedness

Elaboration translates external *Effhole* expressions into the internal syntax described in Figure 5.2. Its main responsibilities are (1) to desugar programmer-facing constructs such as multi-parameter abstractions and `do` blocks into the simpler internal forms, and (2) to decorate every hole with the in-scope variable substitution described in Section 5.2.2.

The algorithm is presented as a function $\mathcal{E}\llbracket e \rrbracket_{\Gamma}$ in Figure 5.3, where Γ is the set of bound variables currently in scope. The top-level elaboration of a closed expression is $\mathcal{E}\llbracket e \rrbracket_{\emptyset}$. For binders, the environment is extended with the newly bound variables before elaborating the subterms; the resulting internal terms are stored as scoped subterms (represented in the implementation by Haskell’s `Bound` library (Kmetz, 2023)), which we write here as $\lambda x. d$, `do  $x \leftarrow d_1$  in  $d_2$` , and so on.

$$\begin{aligned}
\mathcal{E}\llbracket x \rrbracket_{\Gamma} &= x \\
\mathcal{E}\llbracket b \rrbracket_{\Gamma} &= b \\
\mathcal{E}\llbracket n \rrbracket_{\Gamma} &= n \\
\mathcal{E}\llbracket str \rrbracket_{\Gamma} &= str \\
\mathcal{E}\llbracket () \rrbracket_{\Gamma} &= () \\
\mathcal{E}\llbracket e_1 \oplus e_2 \rrbracket_{\Gamma} &= \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma} \oplus \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket [] \rrbracket_{\Gamma} &= [] \\
\mathcal{E}\llbracket e_1 :: e_2 \rrbracket_{\Gamma} &= \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma} :: \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket \lambda x_1 \cdots x_n. e \rrbracket_{\Gamma} &= \lambda x_1. \mathcal{E}\llbracket \lambda x_2 \cdots x_n. e \rrbracket_{\Gamma \cup \{x_1\}} \\
\mathcal{E}\llbracket (e_1, e_2) \rrbracket_{\Gamma} &= (\mathcal{E}\llbracket e_1 \rrbracket_{\Gamma}, \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma}) \\
\mathcal{E}\llbracket \text{inl } e \rrbracket_{\Gamma} &= \text{inl } \mathcal{E}\llbracket e \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket \text{inr } e \rrbracket_{\Gamma} &= \text{inr } \mathcal{E}\llbracket e \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket \text{case } e \text{ of } \{\text{inl } x \mapsto e_1; \text{inr } y \mapsto e_2\} \rrbracket_{\Gamma} &= \text{case } \mathcal{E}\llbracket e \rrbracket_{\Gamma} \text{ of } \{\text{inl } x \mapsto \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma \cup \{x\}}; \text{inr } y \mapsto \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma \cup \{y\}}\} \\
\mathcal{E}\llbracket \text{uncons } e_1 \text{ of } \{[] \mapsto e_2; (x, xs) \mapsto e\} \rrbracket_{\Gamma} &= \text{uncons } \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma} \text{ of } \{[] \mapsto \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma}; (x, xs) \mapsto \mathcal{E}\llbracket e \rrbracket_{\Gamma \cup \{x, xs\}}\} \\
\mathcal{E}\llbracket \text{unpair } e \text{ of } (x, y) \mapsto e' \rrbracket_{\Gamma} &= \text{unpair } \mathcal{E}\llbracket e \rrbracket_{\Gamma} \text{ of } (x, y) \mapsto \mathcal{E}\llbracket e' \rrbracket_{\Gamma \cup \{x, y\}} \\
\mathcal{E}\llbracket \text{handler } \{x \mapsto e; \text{op}_i(x_i, k_i) \mapsto e_i\} \rrbracket_{\Gamma} &= \text{handler } \{x \mapsto \mathcal{E}\llbracket e \rrbracket_{\Gamma \cup \{x\}}; \text{op}_i(x_i, k_i) \mapsto \mathcal{E}\llbracket e_i \rrbracket_{\Gamma \cup \{x_i, k_i\}}\} \\
\mathcal{E}\llbracket e_1 e_2 \rrbracket_{\Gamma} &= \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma} \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \rrbracket_{\Gamma} &= \text{if } \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma} \text{ then } \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma} \text{ else } \mathcal{E}\llbracket e_3 \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket \text{op}(e) \rrbracket_{\Gamma} &= \text{op}(\mathcal{E}\llbracket e \rrbracket_{\Gamma}) \\
\mathcal{E}\llbracket \text{with } e_1 \text{ handle } e_2 \rrbracket_{\Gamma} &= \text{with } \mathcal{E}\llbracket e_1 \rrbracket_{\Gamma} \text{ handle } \mathcal{E}\llbracket e_2 \rrbracket_{\Gamma} \\
\mathcal{E}\llbracket \text{do } \{\bar{s}; e\} \rrbracket_{\Gamma} &= \mathcal{D}\llbracket \bar{s} \rrbracket_{\Gamma}(\mathcal{E}\llbracket e \rrbracket_{\Gamma}) \\
\mathcal{E}\llbracket ?^i \rrbracket_{\Gamma} &= ?^i_{\{x \mapsto x \mid x \in \Gamma\}} \\
\mathcal{D}\llbracket e \rrbracket_{\Gamma}(d) &= d \\
\mathcal{D}\llbracket e; \bar{s} \rrbracket_{\Gamma}(d) &= \text{do } x \leftarrow \mathcal{E}\llbracket e \rrbracket_{\Gamma} \text{ in } \mathcal{D}\llbracket \bar{s} \rrbracket_{\Gamma}(d) \quad (x \text{ fresh}) \\
\mathcal{D}\llbracket x \leftarrow e; \bar{s} \rrbracket_{\Gamma}(d) &= \text{do } x \leftarrow \mathcal{E}\llbracket e \rrbracket_{\Gamma} \text{ in } \mathcal{D}\llbracket \bar{s} \rrbracket_{\Gamma \cup \{x\}}(d) \\
\mathcal{D}\llbracket \text{letrec } f x_1 \cdots x_n = e; \bar{s} \rrbracket_{\Gamma}(d) &= \text{letrec } f x_1 = \mathcal{E}\llbracket \lambda x_2 \cdots x_n. e \rrbracket_{\Gamma \cup \{f, x_1\}} \text{ in } \mathcal{D}\llbracket \bar{s} \rrbracket_{\Gamma \cup \{f\}}(d)
\end{aligned}$$

FIGURE 5.3: The elaboration algorithm from external to internal syntax.

The environment Γ guarantees well-scopedness: every variable that appears free in the internal representation of a subterm is a member of Γ . When elaborating a hole $?^i$, the algorithm builds the identity substitution $\sigma = \{x \mapsto x \mid x \in \Gamma\}$, mapping each in-scope variable to itself. Because Γ is a set, every in-scope variable occurs at most once. Consequently, the algorithm description does not need to handle name shadowing explicitly: the translated internal term preserves the same shadowing behavior as the external term. Because the hole substitution is constructed directly from Γ , any candidate fill for the hole can only refer to variables that were actually in scope at the hole's source location, which is exactly the property required by the safe hole-filling semantics in Section 4.3.2. In the prototype implementation, Γ is represented as an ordered list that also records the source location of each bound variable, so that the interactive environment can highlight the variables captured by each hole context. In that representation, duplicate names are resolved explicitly: when a variable is re-bound, the outer (shadowed) occurrence is discarded and only the most recent binding is retained in the hole substitution.

5.2.4 Evaluation and Simplification Algorithms

We now define the dynamic behavior of *Effhole* by two related algorithms: a deterministic evaluation procedure and a simplification procedure. Both operate on the internal syntax from Figure 5.2. The evaluator reduces a program step by step until it reaches a value or gets stuck on a hole; the simplifier performs additional reduction inside subexpressions and under binders, producing a more aggressively simplified residual program.

Values, indeterminate expressions, and finality. A *value* is a fully evaluated expression. The predicate $value(d)$ holds for variables, literals (booleans, integers, strings, unit), the empty list, abstractions, handlers, and constructed data (lists, pairs, injections) whose components are values. Variables are included as values because the evaluator works on internal terms that may contain free variables, which are treated as irreducible atomic expressions.

A program may also get stuck on a hole before it reaches a value. We call such expressions *indeterminate*. For evaluation, the predicate $indet_E(d)$ is defined recursively in Figure 5.4: a pure hole is indeterminate, an expression is indeterminate when its evaluation is blocked by an indeterminate subterm in a strict position, and a *with* expression is indeterminate when either its handler or its body is stuck in a way that prevents the return clause from applying. An expression is *final* for evaluation, written $final_E(d)$, if it is a value or indeterminate. Evaluation stops exactly at final expressions.

$$\begin{aligned}
 indet_E(?^i_\sigma) &= true \\
 indet_E(d_1 \oplus d_2) &= (final_E(d_1) \wedge indet_E(d_2)) \vee (indet_E(d_1) \wedge final_E(d_2)) \\
 indet_E(\text{with } h \text{ handle } d) &= (final_E(h) \wedge indet_E(d) \wedge \neg retHandled(d)) \vee \\
 &\quad (indet_E(h) \wedge final_E(d)) \\
 retHandled(d) &= value(d) \vee d \equiv ?^u_\sigma
 \end{aligned}$$

FIGURE 5.4: Selected cases of the predicate $indet_E(d)$ for evaluation.

Evaluation contexts. To locate the next subterm to reduce, we use evaluation contexts. An evaluation context E is a term with a single position marker $\square$ that indicates where the next reducible subterm resides. This metavariable $\square$ is distinct from the holes $?^i_\sigma$ that occur in *Effhole* programs; it is merely a placeholder used in the meta-theory of evaluation. The grammar of evaluation contexts is given in Figure 5.5. The constructors enforce a left-to-right, call-by-value evaluation order: for a binary operation $d_1 \oplus d_2$, the left operand is reduced before the right; for application $d_1 d_2$, the function is reduced before the argument; and similarly for other compound expressions. The contexts *WithH* and *WithB* reduce the handler and the body of a *with* expression, and *OpCallE* reduces the argument of an operation call.

We write $E[d]$ for the expression obtained by plugging d into the hole of E . Every context E also denotes a continuation $\text{cont}(E) = \lambda z. E[z]$.

$$\begin{aligned}
E ::= & \square \mid E \oplus d \mid v \oplus E \mid E :: d \mid v :: E \mid \text{uncons } E \text{ of } \{[] \mapsto d_2; (x, xs) \mapsto d_3\} \\
& \mid (E, d) \mid (v, E) \mid \text{inl } E \mid \text{inr } E \mid \text{case } E \text{ of } \{\text{inl } x \mapsto d_1; \text{inr } y \mapsto d_2\} \\
& \mid E d \mid v E \mid \text{if } E \text{ then } d_2 \text{ else } d_3 \mid \text{unpair } E \text{ of } (x, y) \mapsto d_2 \mid \mathbf{op}(E) \\
& \mid \text{with } E \text{ handle } d \mid \text{with } v \text{ handle } E \mid \text{do } x \leftarrow E \text{ in } d_2
\end{aligned}$$

FIGURE 5.5: Evaluation contexts E .

Decomposition. The *decomposition* of a non-final expression d , written $\text{decomp}(d)$, is either undefined or yields a pair (E, r) where E is an evaluation context and r is either a *redex* or an *operation call* located at the hole of E . The redex forms are listed in Figure 5.6. Operation calls $\mathbf{op}(d)$ are not reduced directly; they are handled by a surrounding handler or reported as unhandled if they reach the top level.

The decomposition rules mirror the language structure. For each compound expression, if a subterm is not final, the leftmost non-final subterm is decomposed recursively and its context is wrapped by the appropriate context constructor. When all subterms are final, either a redex is formed or the expression is indeterminate. For example, *uncons* requires a list value; if its scrutinee is a non-list value, decomposition is undefined (a runtime type error).

For *with h handle d*, once h is final the body d is decomposed. If an operation call $\mathbf{op}(v)$ is found in context E inside d , the decomposition checks whether $\mathbf{op}$ belongs to the domain of h and is not captured by an inner handler along E . If both hold, the redex with $h \text{ handle } E[\mathbf{op}(v)]$ is formed; otherwise the operation call is exposed and may escape past the current *with*.

$$\begin{aligned}
r ::= & (\lambda x. d) v_2 \mid v_1 \oplus v_2 \mid \text{uncons } v \text{ of } \{[] \mapsto d_2; (x, xs) \mapsto d_3\} \quad \text{where } v \in \{[], v_1 :: v_2\} \\
& \mid \text{if } b \text{ then } d_2 \text{ else } d_3 \mid \text{unpair } (v_1, v_2) \text{ of } (x, y) \mapsto d_2 \\
& \mid \text{case } v \text{ of } \{\text{inl } x \mapsto d_1; \text{inr } y \mapsto d_2\} \quad \text{where } v \in \{\text{inl } v', \text{inr } v'\} \\
& \mid \text{with } h \text{ handle } E[\mathbf{op}(v)] \mid \text{with } h \text{ handle } v \quad \text{where } v \text{ is final} \\
& \mid \text{do } x \leftarrow v \text{ in } d_2 \mid \text{letrec } f \ x = d_1 \text{ in } d_2
\end{aligned}$$

FIGURE 5.6: Redex forms r .

Reduction. The *reduction* of a redex r , written $red(r)$, is defined as follows. Function application reduces by capture-avoiding substitution: $(\lambda x.d) v \mapsto d[v/x]$. Primitive operators reduce on literals; string concatenation reduces on string literals. The destructors reduce according to the scrutinized value: `uncons` on `[]` returns the nil branch and on $v_1 :: v_2$ substitutes the head and tail into the cons branch; `unpair` and `case` behave analogously; conditionals select the branch determined by the boolean.

Effect handlers are deep. When `with  $h$  handle  $E[\mathbf{op}(v)]$` is reduced, the operation clause for `op` receives v and a continuation $k = \lambda z. \text{with } h \text{ handle } E[z]$ that re-installs the same handler around the rest of the computation. When the body of `with` reduces to a final expression v , the return clause of h is applied to v .

The sequencing construct reduces by `do  $x \leftarrow v$  in  $d \mapsto d[v/x]$` . The recursive definition `letrec  $f\ x = d_1$  in  $d_2$` unfolds once, yielding an abstraction for f whose body contains the original recursive binding.

Evaluation. The *one-step evaluation relation* $d \longrightarrow d'$ holds when $decomp(d) = (E, r)$ and $d' = E[red(r)]$ for a redex r , with operation calls either handled as above or reported as unhandled. The *evaluation* of an expression d is the iterated application of one-step evaluation to a final expression. We write $eval(d) = d'$ when $d \longrightarrow^* d'$ and $final_E(d')$.

For interactive use, evaluation can also be carried out for a bounded number of steps. We write $eval_n(d) = d'$ when d reduces to d' in at most n steps and either d' is final or no further step is possible; if the bound is reached while a step is still possible, the result is undefined.

Because evaluation stops at indeterminate expressions, the result of running a program with holes is already partially simplified: all pure computation that does not depend on a hole is reduced, and the residual expression is either a value or is stuck only because of a hole.

Simplification. The evaluator defined above stops as soon as the whole expression is final, even if subexpressions could still be simplified. For interactive display we define a stronger *simplification* procedure that simplifies subexpressions, including those under binders.

For simplification we use a more liberal finality predicate, written $final_S(d)$. The corresponding indeterminacy predicate $indet_S(d)$ is weaker than $indet_E(d)$: for example, a binary operation or a list construction is indeterminate if *either* subexpression is indeterminate, and a `with` expression is indeterminate as soon as its handler is indeterminate, regardless of the body. These weaker predicates allow simplification to push reduction further inside expressions that are stuck at the top level because of a hole.

The simplification judgment $simpl_n(d) = d'$ is defined recursively on a fuel parameter n :

$$simpl_n(d) = \begin{cases} simpl_{n-1}(d') & \text{if } n > 0 \text{ and } d \longrightarrow_S d' \\ descend_n(d) & \text{if } final_S(d) \\ \text{undefined} & \text{if } n = 0 \text{ and } d \text{ is reducible} \end{cases}$$

where $d \longrightarrow_S d'$ is one-step reduction using the simplification finality predicates. The auxiliary judgment $descend_n(d)$ recursively simplifies the immediate subexpressions of d . Variables and literals have no subexpressions, so $descend_n$ returns them unchanged. For compound expressions such as applications, pairs, lists, injections,

primitive operations, conditionals, operation calls, and with expressions, descend_n applies simpl_n to every immediate subexpression and rebuilds the expression. Binder forms are treated specially, as described next.

For every binder, descend_n chooses a fresh variable, instantiates the body, simplifies it, and re-abstracts the result. Formally, for an abstraction:

$$\text{descend}_n(\lambda x.d) = \lambda x'.\text{simpl}_n(d[x'/x])$$

where x' is fresh with respect to the free variables of d . The same pattern applies to handlers, uncons, unpair, case, and do. This renaming prevents variable capture when hole substitutions contain references to bound variables; the renaming is propagated into the substitutions stored in holes so that captured variables stay consistent.

When simplification reaches a hole $?_\sigma^i$, the hole itself is not reduced, but each expression in the substitution σ is simplified. This ensures that the values captured by a hole are displayed in their simplest form.

Because simplification descends into subexpressions, reduction steps that occur inside a subterm must be lifted to the top-level expression context for display. This is done by applying the appropriate surrounding context constructor to both the expression before the step and the evaluation context recorded for the step. Simplification is also fuel-bounded: the fuel n is decremented on every reduction step, and if it is exhausted while a reducible expression remains, simplification is undefined.

Summary of differences. Evaluation is a left-to-right, call-by-value, outermost reduction strategy: it finds the next redex in the evaluation context and reduces it until the whole expression is final. Simplification uses the same reduction relation but with weaker finality predicates and recursively descends into subexpressions and under binders, producing a more aggressively simplified residual program.

5.2.5 Relation to the Formal Definition of $\lambda_{\text{eff}}^?$

The evaluation and simplification algorithms of *Effhole* are designed to realize the operational semantics of $\lambda_{\text{eff}}^?$ from Section 4.2. In this subsection we make the correspondence explicit and identify where the implementation deviates from the core calculus.

Evaluation as a deterministic strategy. Section 4.2 defines a small-step reduction relation $c \longrightarrow c'$ on computations. The relation is non-deterministic: the ξ rules permit reduction under any structural context, including inside the body of an abstraction or either branch of a do. This non-determinism is convenient for meta-theory but is not directly executable.

The evaluator described above resolves this non-determinism by fixing a left-to-right, call-by-value strategy. An evaluation context E is exactly a particular structural context: it uniquely selects the next redex according to the order in which subexpressions must be evaluated. Consequently, every one-step evaluation $d \longrightarrow d'$ performed by the evaluator is admissible in the non-deterministic semantics of $\lambda_{\text{eff}}^?$ (restricted to the pure-hole fragment and extended with the obvious rules for primitive constructs). The iterated evaluation $\text{eval}(d) = d'$ therefore yields a result that is reachable in the core calculus: $d \longrightarrow^* d'$.

Simplification and structural reduction. The non-deterministic semantics of Section 4.2 also serves as the foundation for simplification. The ξ rules allow reduction under all structural contexts, including under binders. The simplification procedure described above realizes this capability: when no outermost redex exists, it descends into subexpressions and simplifies them, including under binders after a fresh renaming. Thus simplification explores a particular path through the non-deterministic reduction space—one that reduces each subexpression as far as possible.

The two notions of finality reflect two uses of the core semantics. Evaluation uses the stricter predicate $final_E$, which stops at the first indeterminate subterm in a strict position; this corresponds to the intended behavior of running a program, where a hole prevents further execution. Simplification uses the weaker predicate $final_S$, which allows reduction to continue around holes; this corresponds to using the ξ rules to reduce as much as possible even in the presence of holes.

Holes. The most significant difference between $\lambda_{\text{eff}}^?$ and *Effhole* is the treatment of holes. $\lambda_{\text{eff}}^?$ distinguishes pure holes $?_{\sigma}^i$ from impure holes $\{?\}_{\sigma}^i$, whereas *Effhole* exposes only pure holes to the programmer; the rationale for this choice is given in Section 5.2.1. The pure holes of *Effhole*, equipped with their variable substitutions, are exactly the pure holes of $\lambda_{\text{eff}}^?$. Therefore the safety theorem of Section 4.4, that hole filling commutes with reduction, applies directly to *Effhole* programs containing pure holes.

In $\lambda_{\text{eff}}^?$, pure holes are values and may only appear in value positions. Because *Effhole* unifies values and computations, its pure holes may appear in arbitrary positions. The evaluator treats a hole as an indeterminate expression and stops reduction in strict positions around it. For example, an application $?_{\sigma}^i v$ and an operation call $\text{op}(?_{\sigma}^i)$ are both indeterminate and do not reduce further; similarly, a conditional $\text{if } ?_{\sigma}^i \text{ then } d_2 \text{ else } d_3$ cannot select a branch.

5.3 Interactive Features and Language Protocol

The evaluator and simplifier described in the previous section are exposed to programmers through an interactive programming environment. The design keeps the frontend thin: the language machinery resides in the Haskell backend, while the VS Code extension merely sends source code and user actions to the backend and renders the returned structures. This separation keeps the editor plugin small and makes the backend reusable for other editors or web interfaces.

5.3.1 Communication Protocol

In the current prototype the frontend communicates with the backend over HTTP using a simple JSON protocol. The backend exposes a health endpoint for reachability checks and a single run endpoint that accepts a request describing the action to perform, the program text, a fuel bound, and any hole interaction parameters. Responses contain a status code, the computed result, and auxiliary information produced during execution.

The protocol supports five kinds of interaction:

- **Hole discovery.** Before any evaluation, the frontend asks the backend to parse the program and report the positions and identifiers of all holes. This lets the editor decorate hole sites and prepare the user interface for inspection.

- **Evaluation and simplification.** The frontend submits a program together with a fuel bound and receives the final expression produced by the evaluator or simplifier. Fuel limits how many reduction steps may be performed, so diverging programs can be interrupted safely and the server cannot be driven into an unbounded computation.
- **Step-by-step execution.** In addition to the final result, the backend can produce a trace of individual reduction steps. Each step records the redex or operation call being reduced, the expression before and after the step, and the surrounding evaluation context. The frontend uses this trace to animate evaluation inside the editor.
- **Hole inspection.** When the user clicks on a hole in the output panel, the frontend displays the variable context captured at that hole, including the simplified values of in-scope variables and their source locations.
- **Interaction logging.** Simple events such as clicking on a hole are sent to the backend so that user activity can be recorded for later analysis.

The protocol is intentionally narrow. It does not expose the full abstract syntax tree or internal elaboration state; instead, it returns pretty-printed expressions and focused ranges that the frontend can display directly. This keeps the frontend independent of the backend’s internal representation and allows the same backend to serve different editor frontends in the future.

5.3.2 VS Code Extension Frontend

The interactive programming environment is implemented as a VS Code extension for the *Effhole* language. The extension contributes a language mode for files with the `.effhole` extension, basic syntax highlighting, and a small set of user-facing commands. The main commands let the user run the current file, toggle between evaluation and simplification modes, simplify a selected expression, and step through evaluation one reduction at a time.

When the user invokes a run command, the extension saves the active editor, sends the program text to the backend, and displays the result in a dedicated web-view panel beside the editor. The panel has two regions. The upper region shows the final expression returned by the backend, or, in step mode, the expression at the currently viewed reduction step. The lower region, labelled “Hole Contexts,” shows the variable bindings captured at holes that the user has chosen to inspect.

Several features make the output panel interactive:

- **Hole identifiers as inlay hints.** The extension registers an inlay-hint provider that asks the backend for the positions of all holes and renders each hole’s stable identifier next to its source occurrence. This helps the programmer correlate holes in the source with holes in the output panel.
- **Clickable holes.** Hole occurrences in the output are rendered as clickable links. Clicking a hole pins its context to the “Hole Contexts” region, where each variable is shown together with its simplified value. The most recently inspected contexts are kept at the front of a small queue, while older ones are dropped according to a configurable window size.
- **Hover-to-definition.** Hovering over a captured variable in a hole context highlights the variable’s definition range in the source editor and scrolls it into view.

if necessary. Similarly, hovering over a pinned hole card highlights the corresponding hole site in the source. These decorations give the programmer immediate visual feedback about where each captured variable was bound.

- **Step navigation.** In step mode, the output panel displays previous and next buttons and a step counter. As the user moves through the trace, the currently focused redex or operation call is highlighted in the displayed expression, and the hole contexts are updated to match the state before the current step.

The extension also performs a health check against the backend before issuing requests. If the backend cannot be reached, the user is shown a diagnostic message that points to the server URL setting. Settings such as the backend URL, request timeout, default fuel, and the number of displayed hole contexts are configurable.

5.3.3 Data Collection for Interactive Features

The backend also collects data about how users interact with *Effhole* programs, in order to support the evaluation in Chapter 6 and to inform future research on programming with holes and effects. The collected data is designed to answer questions such as which holes users inspect, how often users interact with holes, and what variable contexts are available at those holes. All collection happens on the backend so that the frontend remains simple and the data format is stable across editor versions.

Every request received by the language server is persisted to a local log file in JSON Lines format. The log records the raw request, including the command, the program text, the fuel bound, the target hole name and context when applicable, and optional metadata such as a username and timestamp. The frontend supplies a username from its configuration and a timestamp from the system clock. This request-level logging captures a complete trace of user activity without requiring any instrumentation inside the evaluator.

In addition to request logging, the backend extracts structured information about holes whenever a program is evaluated or simplified. Each hole occurrence is assigned a stable identifier so that the same hole can be recognized across multiple interactions. During execution, the backend counts how many times each hole is reached and records the substitution associated with every reached occurrence. This substitution contains the values of all variables that were in scope at the hole, together with their source positions. Because the values are simplified before being stored, the log reflects the simplest form of each captured variable at the moment the hole was encountered.

Hole clicks are logged explicitly through a dedicated protocol command. When the user clicks a hole in the output panel, the frontend sends the hole name and its current context to the backend, which appends the event to the request log. This makes it possible to relate the static occurrence of a hole in the source code to the dynamic contexts in which it was inspected.

Chapter 6

Evaluation

Having presented the formalization and implementation of holes for effect handlers, we now evaluate whether this design helps programmers understand effect handlers in practice. The preceding chapters introduced $\lambda_{\text{eff}}^?$, a core calculus that justifies fill-and-resume soundness for both pure and impure holes, and *Effhole*, a prototype programming environment that exposes these ideas through an interactive VS Code extension. The design hypothesis is that allowing programs containing holes to execute, while displaying the captured continuation states and simplifying complex residuals, reduces the cognitive load of learning effect handlers. This chapter tests that hypothesis through a small-scale qualitative user study.

We first state the research questions that motivate the study and then describe our method: the participants, the study design, the six programming tasks of increasing difficulty, and the qualitative analysis approach (Section 6.2.4). We then present the results, organized around each research question, together with participant feedback that points to concrete directions for improvement. Finally, we discuss what the findings imply for positioning holes as a learning scaffold and for the broader significance of simplification in interactive programming with effect handlers. Because the study is exploratory and the sample is small, the goal is to identify promising design directions and limitations rather than to establish generalizable performance claims.

6.1 Research Questions

Our hypothesis is that allowing execution of programs containing holes can expose intermediate continuation states and reduce cognitive load, and that simplification can reduce the complexity of the results. This chapter evaluates that hypothesis through a user study. We aim to answer the following research questions:

- **RQ1:** Do holes help programmers understand the behavior of effect handlers by enabling them to inspect values and continuations at runtime?
- **RQ2:** Does simplification improve the readability of the result, especially for continuations?
- **RQ3:** How does prior experience with effect handlers and Agda influence the usefulness of holes?

6.2 Method

We conducted a small-scale user study to gain qualitative insights into how programmers interact with holes, rather than to establish generalizable performance differences.

We designed 6 programming tasks of increasing complexity, targeting different usage scenarios of effect handlers and holes. During the study, we collected qualitative data through operation recordings of participants' interactions with the tool and follow-up interviews to understand their thought processes and feedback.

6.2.1 Participants

The target users of our tool were programmers who are familiar with functional programming but have varying levels of experience with effect handlers, ranging from effect handler novices to advanced users. We recruited 4 participants from our institution, categorized as three undergraduate students and one graduate student.

Table 6.1 summarizes their relevant prior experience and self-rated familiarity with effect handlers and holes. Participants' backgrounds varied both in their exposure to Agda-style interactive proof assistants and in their understanding of effect handlers. U1 and G1 had read Agda tutorials but had limited hands-on experience with the environment. U2 and U3, in contrast, had also used Agda more and were comfortable with its interactive workflow, including the use of holes. Regarding effect handlers, U1, U2, and U3 had gained a basic understanding from tutorial materials; G1 had gone further and had previously implemented a language with effect handlers in C++, giving them a comparatively deep knowledge of the mechanism. In addition, G1 described themselves as an experienced imperative programmer with a preference for statically typed languages. This diversity allowed us to explore how holes support understanding across different experience levels.

Participant	Agda Experience	EH Familiarity
U1	Basic	Basic (self-rated 10%)
U2	Familiar	Basic
U3	Familiar	Basic
G1	Basic	High; EH implementation in C++

TABLE 6.1: Overview of study participants and their prior experience with effect handlers.

6.2.2 Study Design

We adopted a qualitative study design aimed at understanding how programmers interact with holes and simplification in realistic problem-solving situations.

Participants were paired for scheduling purposes (U1 with U2, and U3 with G1), but each participant completed the same tasks independently. Each session lasted approximately 70 minutes, conducted using the participant's own laptop with the *Effhole* language extension installed in Visual Studio Code. Using participants' own machines ensured that they interacted with the tool in an environment close to their everyday programming workflow.

Each session consisted of three phases. First, a warm-up phase (approximately 5 minutes) introduced the core language constructs used in the tasks and the extension's interface. The experimenter showed a tutorial covering the basic syntax and demonstrated how to execute a program, how to inspect the context displayed for a hole, and how to switch between evaluation and simplification modes. Participants were encouraged to ask questions during this phase and to try out the environment themselves before beginning the tasks.

The main task phase lasted 45–60 minutes. Participants worked through Tasks 1 to 6 in the fixed order shown in Table 6.2, with the difficulty increasing gradually. They were free to use any feature of the tool, including inserting additional holes, switching between evaluation and simplification modes, and re-running programs as often as they wished. The experimenter intervened only to clarify task requirements or to give help if a participant was completely stuck for more than 5 minutes. After each task, participants were asked some brief questions about their strategy and experience with that task before moving on to the next one.

After completing the tasks, participants took part in a short interview (approximately 10 minutes). The interview focused on their overall strategy for each task, when and why they used holes, which representations they found helpful or confusing, and any suggestions for improving the tool.

All sessions and interviews were conducted in Japanese. Participant quotes reported in this chapter have been translated into English by the author for presentation.

6.2.3 Task Design

Design Principles The tasks were designed to observe how the tool supports understanding at different stages of learning effect handlers. Each task targets a specific usage scenario, with increasing complexity and progressively less explicit guidance. We varied two factors systematically across the tasks: whether holes were pre-placed by the experimenter or left to participants to insert freely, and how much the task was expected to benefit from simplification.

Pre-placed Holes vs. Free Insertion The distinction between pre-placed and freely inserted holes was central to the design. Pre-placed holes reduce initial friction, letting participants experience the value of hole contexts before they try their own exploration. Free insertion, by contrast, reveals whether participants adopt holes as an active strategy once scaffolding is removed. We used this contrast to investigate when participants know where to place holes and when they do not.

Importantly, this distinction reflects only whether the expected solution would need participants to insert holes themselves, but not any restriction on their behavior. Participants were free to insert, remove, or modify holes in every task, and observing how they spontaneously used holes under different scaffolding conditions was itself an objective of the study.

Task Overview Table 6.2 summarizes the six tasks. The progression begins with exercises that familiarize participants with the hole-context format, then moves to handler implementation, and finally to open-ended tasks that require reasoning about nested or conditional continuations with little explicit guidance.

Task 1 serves as a warm-up: participants encounter a hole in a pure program with no handlers, allowing them to learn the basic structure of the hole context without the added complexity of continuations. This separation is important because, unlike Agda-style holes that mainly expose types and local variables, *Effhole* contexts also represent captured continuations when dealing with effect handlers. Isolating the interface in a pure program lets participants first understand what the tool displays before confronting complex continuation structure.

Task 2 introduces a simple effect handler and asks participants to observe how the hole is included in the result. Task 3 then asks them to implement a similar

Task	Core Concept	Pre-placed Holes?	Simplification Expected?	Difficulty
Task 1	Simple task with holes (no handlers)	Yes	No	Very easy
Task 2	Writer effect, observation only	Yes	No	Easy
Task 3	Writer handler implementation	No	No	Easy
Task 4	Exceptions	No	Yes	Medium
Task 5.1	State handler, observation only	No	Yes	Hard
Task 5.2	State handler, implementation	Yes	Yes	Hard
Task 6	Nondeterminism, little guidance	No	Yes	Hard

TABLE 6.2: Overview of study tasks, their target concepts, hole placement, expected need for simplification, and difficulty.

handler without pre-placed holes; they must insert holes themselves or build on what they learned in Task 2.

Task 4 presents a handler with conditional branching and asks participants to decide where to place holes to explore both branches. This task requires understanding continuations by exploring hole contexts.

Task 5 concerns a state handler implemented in parameter-passing style, where the handler returns a function and simplification becomes essential for reading the result and the continuation in the context. The task is split into an observation sub-task (Task 5.1), in which participants freely insert holes to explore the code, and an implementation sub-task (Task 5.2), in which they complete a similar handler with pre-placed holes.

Finally, Task 6 presents the most open-ended scenario, requiring participants to reason about a complex continuation with minimal guidance.

6.2.4 Data Collection & Analysis

Data Sources As described in Section 5.3.3, we use the *Effhole* extension to collect data about participants’ interactions with the tool. We collected two types of data during each session. First, operation recordings captured participants’ interactions with the *Effhole* extension, including how they switched between evaluation and simplification modes. The frequency with which they used these modes can be inferred from these recordings. The tool also collected the source code attempted by participants, including intermediate and final versions. These files helped us reconstruct the evolution of each participant’s approach and how they used holes. These automatic recordings provided a detailed trace of tool usage without disrupting the participants’ workflow. Second, the post-task interview recordings captured participants’ reflective accounts of their strategies, confusion points, and suggestions for improvement. Together, these sources allowed us to observe a comprehensive picture of how participants interacted with the tool and their subjective experiences.

Analysis Methods Given the small sample size ($n = 4$), we did not perform statistical inference or compare group means. Instead, we analyzed the data qualitatively, treating each participant as an individual case study. For each participant, we identified themes that recurred across the sessions, such as strategies for using holes, situations in which simplification became necessary, and common sources of confusion. At the same time, we remained attentive to individual differences arising from participants’ prior experience with effect handlers. Findings from this analysis are intended to identify opportunities and limitations in the current design rather than to establish general conclusions.

6.3 Results

6.3.1 Task Completion and Hole Usage Overview

Task Completion Within the allotted study time, only G1 completed all six tasks, and did so more quickly than the other participants. This is consistent with G1’s prior experience implementing effect handlers and their self-rated high familiarity: in the post-study interview they reported that they was already comfortable reasoning about handler behavior statically. The remaining participants did not finish all tasks: U1 and U2 were unable to complete Task 5.1, Task 5.2 and Task 6, while U3 became stuck on Task 5.2 and unable to proceed.

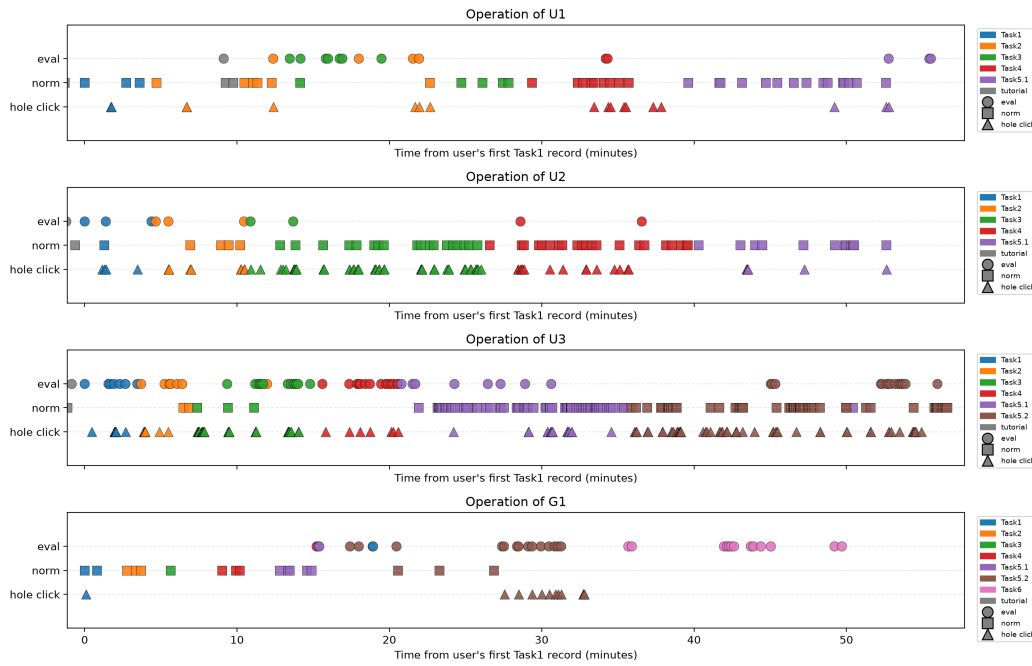


FIGURE 6.1: Overview of task completion and hole usage across participants.

Operation Sequences and Tool Usage Frequency Based on the operation recordings, Figure 6.1 shows the sequence of tool interactions for each participant during the session. The horizontal axis represents elapsed time aligned to the beginning of Task 1. Each task is represented by a distinct color, and each recorded operation is marked by a shape: circles denote running a program in evaluation mode, squares denote running in simplification mode, and triangles denote viewing a hole context.

The figure reveals large differences in how actively participants used the tool. U2 and U3 interacted with the tool most frequently, and they viewed hole contexts repeatedly throughout the session. This pattern likely reflects their prior experience with Agda, which follows a similar interactive workflow of refining programs around holes. U1 used the tool somewhat less often and tended to run programs and inspect the final result directly, rather than consulting the hole context as frequently. G1 used the tool least of all. They consulted a hole context only during Task 5.2, and otherwise typically ran a program once and inferred the answer directly from

the output. This aligns with their interview comment that they prefer to derive program behavior statically in their head, and that type information is more helpful to them than dynamic execution results.

Simplification and Task Difficulty Looking at the distribution of operation types, the data for U1, U2, and U3 all show that the frequency of simplification-mode runs increased markedly as the tasks became harder, especially in Task 5. This trend was also observed in the interviews: participants reported turning to simplification when the raw evaluation output became too verbose to read. In contrast, G1’s comparatively sparse tool use meant that they rarely needed simplification at all.

6.3.2 Holes Support Program Understanding (RQ1)

Our first research question was: *Do holes help programmers understand the behavior of effect handlers?* According to the interviews, participants reported that holes helped them infer handler implementations by exposing intermediate continuations and values, particularly when the continuation structure was not immediately obvious from the source code.

Holes as “Program Inspector” (Task 3 / U2) In Task 3, U2 inferred the handler implementation directly from the pair structure shown in the hole context (Figure 6.2), without needing to step through the source code mentally. They commented, “*I only looked at the hole context on the right and solved the task. I didn’t even really need to think much.*” This indicates that holes **externalize** the execution process that normally requires mental simulation, making intermediate states visible: participants do not need to mentally run the program; they can “read” the current shape of the continuation directly, shifting the working-memory burden to an external visualization.

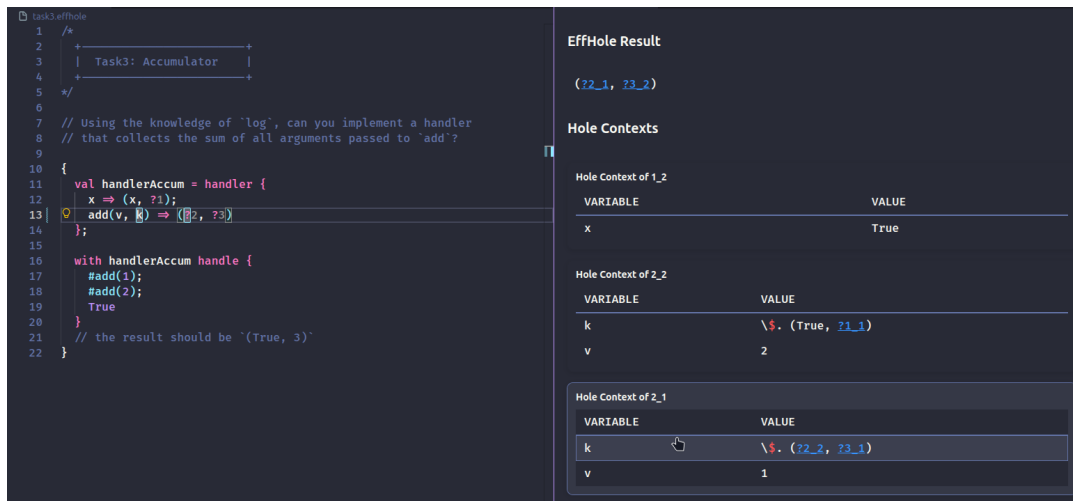


FIGURE 6.2: The hole U2 inserted in Task 3

Holes as Runtime Probes (Task 4 / U3) U3 actively used `if b then ? else ?` in Task 4 to observe program behavior under the different branches. This illustrates the role of holes as **runtime probes**: unlike Agda holes, which are type-guided, *Effhole* holes support runtime behavioral exploration.

Holes Preserve Intermediate Results (Task 2 / U1) U1 noted that the hole context let them see intermediate results and observe the string concatenation result without complete understanding of the continuation. This suggests that holes can help users implement programs with effect handlers even when they do not fully understand the continuation structure in the handler.

Summary The core finding for RQ1 is that holes are most valuable when the continuation structure is non-trivial. However, displaying the hole context alone is not enough; simplification is needed to make the result readable (RQ2).

6.3.3 Simplification Supports Continuation Understanding (RQ2)

Does simplification encourage participants to use the hole context more effectively?

Simplification is Necessary in Complex Code (Task 5.1 / U3) In the nested recursion of state handler, the evaluation-mode output was filled with verbose lambda abstractions, making it difficult to identify the relevant continuation structure. U3 explicitly stated that: *“It was unreadable without simplification.”* This suggests that, while eval mode suffices for simple tasks (Task 1–Task 3), simplification becomes a prerequisite for understanding in nested handlers such as Task 5.

Simplification is Tightly Coupled with Hole Usage (G1) Because G1 did not need to inspect complex continuations, they had little opportunity to experience the benefit of simplification. This indicates that the value of simplification emerges primarily when one needs to deeply inspect continuations.

Summary Simplification was most valuable in tasks where participants made the most use of holes (Task 5), suggesting that the two features are designed to be used together.

6.3.4 Individual Differences and Experience Level (RQ3)

How does prior experience with effect handlers influence the usefulness of holes?

According to the interviews, participants’ prior experience with effect handlers strongly influenced how they used holes and how much they benefited from them.

Participant with High Familiarity (G1) As already seen in the operation-sequence analysis, G1, who had prior experience implementing effect handlers in C++, did not use holes frequently and relied instead on type systems and mental models to infer implementations directly. They explained, *“I rely more on types than on execution.”* For programmers with a strong mental model, the execution information provided by holes is redundant. This also reflects the fact that the tasks were relatively simple; exploring more advanced use cases remains future work.

Participants with Basic or Low Familiarity (U1, U2, and U3) U2 and U3 both had prior experience with Agda and used holes frequently, though they differed in how much they benefited.

U1 (Agda experience, self-rated 10% effect-handler familiarity) used holes moderately and benefited most from Task 2, where the continuation visualization was simple. However, they reported that they did not fully understand continuations

throughout the tasks and found complex programs hard to read. This suggests that holes can support even those with minimal effect-handler understanding, but more complex cases still require basic knowledge of continuations.

U2 (Agda experience, prefer hole interaction) used holes with high frequency and benefited most from Task 3, where they solved the task directly from the hole context (Section 6.3.2). They got stuck on Task 4 and Task 5, noting that *“rather than helping me understand from 0 to 1, I think it helps me organize things already in my mind.”* Thus, for simple tasks, users who are familiar with holes can take advantage of the visualization; but for more complex tasks, they still need to understand the continuation to make sense of the hole context.

U3 (Agda and STLC proof experience) also used holes frequently and explored actively, for example by using `if b then ? else ?` in Task 4. They benefited most from Task 4, where holes reduced cognitive effort, but found Task 5 challenging even with holes and simplification, commenting that it is *“hard to understand when you don’t know the programming paradigm.”*

Summary Holes can support programmers with varying levels of experience, but they are most useful as a learning scaffold for those who are still building a mental model of effect handlers, such as programmers with functional-programming experience but limited effect-handler background. For complete novices, additional guidance may be needed to interpret the hole context.

6.3.5 User Feedback and Improvement Suggestions

Although holes provided significant support for novices, participant feedback also revealed key limitations in the current design. These limitations are unsolved problems in the design space rather than flaws of the study itself.

Difficulty Deciding Hole Insertion Locations Participants found it easier to work with tasks that already contained pre-placed holes. For example, U2 noted that *“the tasks I found easy were the ones that already had holes prepared for me.”* In tasks requiring free hole insertion (Task 3, Task 4, Task 5.1, and Task 6), participants needed to decide where to place holes themselves, which added an additional planning burden.

Hole IDs and Nesting Complexity U3 reported that hole IDs such as `?1, ?2, ...` became confusing when continuations were nested. In Task 5, multiple nested recursive holes made it difficult to know which hole’s context was currently being inspected. This suggests that linear hole IDs are insufficient to express structural relationships in deeply nested structures. Possible directions include better visualization (e.g., a tree structure) or more informative hole IDs. In addition, G1 suggested that it would be helpful to enable users to name holes.

Need for a Type System U3 explicitly stated that a type system would help them decide what to fill in a hole, and G1’s feedback also centered on type-driven development. The current prototype lacks type information, so holes can answer *“what is the continuation?”* but not *“what type of argument is required by the continuation?”*

Moreover, we observed that participants struggled to understand what is expected of continuation arguments, and they often had to guess what the correct type should be. Integrating type information would therefore address a clear need expressed by participants.

6.4 Discussion

Evaluating the Practical Utility of the Formalization The positive findings from our user study provide initial support for the practical value of our formalization, in particular the semantics of holes and simplification. Participants were able to use these mechanisms to make intermediate continuation states visible and manageable, suggesting that the design of our formalization has meaningful consequences for interactive programming tools.

Positioning Holes as a Learning Scaffold The RQ3 findings suggest that *Effhole* is best positioned for programmers currently learning effect handlers, bridging the gap between “complete novice” and “established mental model.” For this audience, holes provide external support that compensates for an incomplete mental model, while more experienced programmers may rely on static reasoning and type information instead. This boundary is captured by U2’s remark that holes helped them “organize things already in my mind” rather than “understand from 0 to 1”, indicating that holes are most useful when learners already have a partial intuition and need help making the continuation structure concrete, not when they lack any prior understanding of effect handlers.

Design Significance of Simplification As the complexity of continuations increases, the cognitive load of the raw evaluation representation grows quickly. Simplification is therefore essential for keeping hole contexts readable when continuations become deeply nested.

Chapter 7

Related Work

This chapter discusses related work on programming with holes, reduction theory of effect handlers, IDE support for effect handlers.

Programming with Holes Hazel (Omar et al., 2019) is a live programming environment with typed holes and real-time feedback. It is purely functional and does not support computational effects. In contrast, *Effhole* is designed to support computational effects through the use of algebraic effects and handlers. This allows *Effhole* to model a wider range of programming paradigms, including imperative and concurrent programming. In addition, the metatheory of Hazel is based on a call-by-value small-step semantics, while *Effhole* is based on a full reduction of a fine-grain call-by-value semantics. This allows *Effhole* to establish a more general version of confluence, and the soundness of the fill-and-resume operation. Hazel also offers richer live features such as live pattern matching (Yuan et al., 2023) and live type error localization (Zhao et al., 2024); these are orthogonal to the core contributions of this thesis and could be added to *Effhole* in future work.

Agda (Norell, 2009) provides an interactive, type-directed editing environment in which users can leave typed holes and use commands such as refinement, case splitting, and goal normalization. As noted in Section 5.2.1, the current *Effhole* prototype exposes only pure holes because it cannot reliably check whether a fill is a value or a computation. An Agda-style interaction, in which the system checks the type and effect of the fill and either accepts it and resumes evaluation or rejects it with a reason, would enable a more complete tool. Such a mechanism could support both pure and impure holes while guaranteeing that each hole is filled only with expressions that satisfy its contract.

Reduction Theory of Effect Handlers The operational semantics of effect handlers found in most practical programming language implementations is typically formulated as a call-by-value semantics geared towards efficient execution. By contrast, the semantics we present in Section 4.2 is a full reduction system in which every redex may be reduced at any context inside a term, and we prove that this system enjoys confluence.

Sieczkowski, Pyzik, and Biernacki, 2023 also develop a full reduction theory for deep effect handlers. Their calculus, however, is deliberately designed around a set of non-standard, fine-grained rewrite rules that facilitate compiler optimization and equational reasoning about programs. Our work starts from a different goal: we want a semantics that makes programs as small and understandable as possible, so we adopt a more conventional rewriting strategy based on standard fine-grain call-by-value reduction rules. This choice keeps the reduction relation close to ordinary equational simplification and is better suited to the interactive programming setting of *Effhole*.

IDE Support for Effect Handlers Neumann, 2021 explores how effect information can be used directly in an IDE. In the Effekt language and its Visual Studio Code plugin, inferred effect types appear in hover tooltips and inline annotations, effect operations are navigable like function calls, and the editor can scaffold handler templates around selected expressions. This work is complementary to *Effhole*: while we focus on live evaluation around holes, Neumann’s features address the orthogonal problems of reading, navigating, and refactoring effectful code. Integrating analogous effect tooltips, navigation, and handler scaffolding into a future *Effhole* IDE would make the effect information more visible and lower the cost of constructing handlers.

Chapter 8

Conclusion

8.1 Conclusion

In this thesis, we addressed the problem of understanding and constructing programs that use algebraic effect handlers by integrating holes into the language.

We first identified a key semantic challenge: in the presence of effect handlers, naively allowing evaluation to proceed around holes can make the order of effects observable, breaking the fill-and-resume safety property that underpins interactive programming with holes. To solve this problem, we developed $\lambda_{\text{eff}}^?$, a fine-grain call-by-value calculus with deep effect handlers and both pure and impure holes. We gave a small-step operational semantics that restricts effect propagation across unfilled holes, and we proved the key meta-theoretic properties of commutativity and confluence. Together, these properties imply a general soundness theorem for fill-and-resume.

A central insight is that the distinction between pure and impure holes reflects a fundamental difference in what is known about the expression that will fill the hole: a pure hole is guaranteed to be filled with an effect-free value, whereas an impure hole may be filled with an arbitrary computation. Fine-grain call-by-value makes this distinction syntactically explicit, and our semantics uses it to control the order of effects precisely, allowing evaluation to proceed around impure holes when the observable effect order is preserved. Because our soundness theorem holds for both kinds of holes, an implementation may expose either one or both without sacrificing correctness, leaving the surface-language design as a flexible choice.

Building on this formalization, we implemented *Effhole*, a prototype programming environment realized as a VS Code extension backed by a Haskell interpreter and language server. *Effhole* provides deterministic evaluation, aggressive simplification, and interactive inspection of hole contexts, making the intermediate states of effectful computations visible to programmers.

We evaluated the design through a small-scale user study. The results suggest that holes are most helpful as a learning scaffold for programmers who are still building a mental model of effect handlers, and that simplification becomes essential when continuations are deeply nested. The study also identified directions for improvement, including richer hole visualization, user-named holes, and type-driven hole filling.

8.2 Future Work

Type System Integration As noted in the user feedback results, the current prototype lacks type information, limiting type-driven hole filling. A natural future direction is to integrate a type system so that the hole context displays both the expected

type and the current runtime value.

Formalization of Deterministic Evaluator The operational semantics of $\lambda_{\text{eff}}^?$ is non-deterministic: it permits reduction under every syntactic context, including under binders. The implementation of *Effhole*, by contrast, uses deterministic evaluation and simplification procedures to make the tool predictable and efficient. Although the implementation is designed to mirror the formal semantics, we have not proved a formal correspondence between the non-deterministic reduction relation and the deterministic evaluator. Future work should establish such a correspondence—for example, by showing that deterministic evaluation and simplification procedure preserves the equational theory induced by the reduction relation. Such a proof would strengthen the trustworthiness of the implementation.

Comparison with Traditional Debugger Breakpoints Traditional breakpoints pause at fixed locations, while holes may not necessarily block execution. The behavior of holes is more flexible, depending on how the semantics is designed. The best design choice may depend on the intended use and is an open question for future work.

User Studies for Advanced Use Cases As noted in Section 6.2.4, the small sample size limits statistical inference. Additional methodological limitations include the absence of a control group (e.g., a “no holes, plain text editor” condition), which makes it difficult to separate tool effects from individual learning effects, and the fact that participants were recruited from a single institution, which may introduce selection bias. Future work should target more advanced use cases, such as complex real-world programs and debugging scenarios.

Bibliography

- Bingham, Eli et al. (Jan. 2019). “Pyro: deep universal probabilistic programming”. In: *J. Mach. Learn. Res.* 20.1, 973–978. ISSN: 1532-4435.
- Garner, Richard (2009). “On the strength of dependent products in the type theory of Martin-Löf”. In: *Annals of Pure and Applied Logic* 160.1, pp. 1–12. ISSN: 0168-0072. DOI: <https://doi.org/10.1016/j.apal.2008.12.003>. URL: <https://www.sciencedirect.com/science/article/pii/S0168007208001954>.
- Hillerström, Daniel and Sam Lindley (2018). “Shallow Effect Handlers”. In: *Programming Languages and Systems*. Ed. by Sukyoung Ryu. Cham: Springer International Publishing, pp. 415–435. ISBN: 978-3-030-02768-1.
- Kammar, Ohad, Sam Lindley, and Nicolas Oury (Sept. 2013). “Handlers in action”. In: *SIGPLAN Not.* 48.9, 145–158. ISSN: 0362-1340. DOI: [10.1145/2544174.2500590](https://doi.org/10.1145/2544174.2500590). URL: <https://doi.org/10.1145/2544174.2500590>.
- Kmett, Edward A. (2023). *bound: Combinators for Manipulating Locally-Nameless Generalized de Bruijn Terms*. Haskell package, version 2.0.7. URL: <https://hackage.haskell.org/package/bound>.
- Levy, PaulBlain, John Power, and Hayo Thielecke (2003). “Modelling environments in call-by-value programming languages”. In: *Information and Computation* 185.2, pp. 182–210. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/S0890-5401\(03\)00088-9](https://doi.org/10.1016/S0890-5401(03)00088-9). URL: <https://www.sciencedirect.com/science/article/pii/S0890540103000889>.
- Neumann, Tim (2021). *IDE Support for Lexical Effects and Handlers*. Blog post summarizing the bachelor’s thesis. URL: <https://se.informatik.uni-tuebingen.de/teaching/thesis/2021/11/01/IDE-effects/>.
- Norell, Ulf (2009). “Dependently typed programming in Agda”. In: *Proceedings of the 4th International Workshop on Types in Language Design and Implementation*. TLDI ’09. Savannah, GA, USA: Association for Computing Machinery, pp. 1–2. ISBN: 9781605584201. DOI: [10.1145/1481861.1481862](https://doi.org/10.1145/1481861.1481862). URL: <https://doi.org/10.1145/1481861.1481862>.
- Omar, Cyrus et al. (Jan. 2019). “Live functional programming with typed holes”. In: *Proc. ACM Program. Lang.* 3.POPL. DOI: [10.1145/3290327](https://doi.org/10.1145/3290327). URL: <https://doi.org/10.1145/3290327>.
- Pierce, Benjamin C. (Feb. 2002). *Types and Programming Languages*. 1st ed. MIT Press. ISBN: 0262162091. URL: http://ropas.snu.ac.kr/~kwang/520/pierce_book.pdf.
- Plotkin, Gordon and John Power (2003). “Algebraic operations and generic effects”. English. In: *Applied Categorical Structures* 11.1, pp. 69–94. ISSN: 0927-2852. DOI: [10.1023/A:1023064908962](https://doi.org/10.1023/A:1023064908962).
- Plotkin, Gordon D and Matija Pretnar (Dec. 2013). “Handling Algebraic Effects”. In: *Logical Methods in Computer Science* Volume 9, Issue 4. DOI: [10.2168/LMCS-9\(4:23\)2013](https://lmcs.episciences.org/705). URL: <https://lmcs.episciences.org/705>.
- Sieczkowski, Filip, Mateusz Pyzik, and Dariusz Biernacki (Aug. 2023). “A General Fine-Grained Reduction Theory for Effect Handlers”. In: *Proc. ACM Program. Lang.* 7.ICFP. DOI: [10.1145/3607848](https://doi.org/10.1145/3607848). URL: <https://doi.org/10.1145/3607848>.

- Sivaramakrishnan, KC et al. (2021). “Retrofitting effect handlers onto OCaml”. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2021. Virtual, Canada: Association for Computing Machinery, 206–221. ISBN: 9781450383912. DOI: [10.1145/3453483.3454039](https://doi.org/10.1145/3453483.3454039). URL: <https://doi.org/10.1145/3453483.3454039>.
- Takahashi, M. (1995). “Parallel Reductions in λ -Calculus”. In: *Information and Computation* 118.1, pp. 120–127. ISSN: 0890-5401. DOI: <https://doi.org/10.1006/inco.1995.1057>. URL: <https://www.sciencedirect.com/science/article/pii/S0890540185710577>.
- Yuan, Yongwei et al. (Apr. 2023). “Live Pattern Matching with Typed Holes”. In: *Proc. ACM Program. Lang.* 7.OOPSLA1. DOI: [10.1145/3586048](https://doi.org/10.1145/3586048). URL: <https://doi.org/10.1145/3586048>.
- Zhao, Eric et al. (Jan. 2024). “Total Type Error Localization and Recovery with Holes”. In: *Proc. ACM Program. Lang.* 8.POPL. DOI: [10.1145/3632910](https://doi.org/10.1145/3632910). URL: <https://doi.org/10.1145/3632910>.

Appendix A

Source Code for User Study

```

/*
+-----+
| Task1: Average score |
+-----+

This is a program that computes the weighted average score
of a list of students.
The weight for midterm and final are 30% and 70% respectively.

Please try to run the program and answer:
  1. What is the problem with the original code?
  2. What should be put in place of '?1' ?
*/

{
  def map(f)(lst) = lst match {
    [] => []; x :: xs => f(x) :: map(f)(xs)
  };

  val students =
    ("Alice", (88, 89))
    :: ("Bob"  , (92, 85))
    :: ("Chris", (85, 90))
    :: []; // student name and their midterm and final scores

  val weightedAvg = \s.
    s match {
      (name, scores) => scores match {
        // compute the weighted average score
        (mid, fin) => mid * 30 + ?
      }
    };

  // compute the weighted average score for each student
  map(weightedAvg)(students)
}

/*
+-----+
| Task2: Logging      |
+-----+
*/

{
  // The first effect we introduce is 'log'.

```

```

// Similar to how we handle exceptions in conventional
// programming languages, we can use effect handlers
// to handle 'log':
val handleLog = handler {
  x => (x, "");
  log(s, k) => k(()) match {
    (x, acc) => (x, s ^ acc)
  }
};

// This handler extends the resulting value with an empty
// string. When a 'log' call is encountered, we resume
// the continuation, obtain the accumulated string 'acc',
// and append the parameter 's' to it to produce the final result.

// We can observe the behavior of this handler more clearly by
// applying it to a concrete computation with holes:
with handleLog handle {
  #log(?);
  #log(?);
  ()
}

/*
The resulting string is '?1 ^ (?2 ^ "")',
which represents the concatenation of '?1' and '?2'.
Please write down the result of the following program:

with handleLog handle {
  #log("hello, ");
  #log("world");
  ()
}

*/

/*
+-----+
| Task3: Accumulator |
+-----+
*/

// Using the knowledge of 'log', can you implement a handler
// that collects the sum of all arguments passed to 'add'?

{
  val handlerAccum = handler {
    x => (x, 0);
    add(v, k) => k(()) match {
      (x, acc) => (x, v + acc)
    }
  };

  with handlerAccum handle {
    #add(1);
    #add(2);
    True
  }
}

```



```

    }
    // the result should be '(True, 3)'
}

/*
+-----+
| Task4: Assertion      |
+-----+
*/

// We can do something more interesting with continuations.
// To achieve this, we can start with a handler
// containing a hole:

{
    // For example, we can implement the operation 'assert(b)':
    // it does nothing if 'b' is true, and skips the remaining
    // computation if 'b' is False.
    // We know that the factorial function does not work if
    // its argument is less than 0.
    // We can detect this using 'assert':
    def fact(x) = {
        #assert(x >= 0);
        if x == 0 then 1 else x * fact(x - 1)
    };

    val handlerAssert = handler {
        x => x;
        assert(b, k) => if b then k(()) else "error"
    };

    with handlerAssert handle {
        fact(-1)
    }
}

/*
+-----+
| Task5_1: State        |
+-----+
*/

// We can also implement mutable state using effect handlers.
// Here is an example that counts how many times a function is called:

{
    val compute = \x. {
        #incr(());
        x * 2
    };

    // We can implement this using effect handlers
    // This definition can be difficult to understand.
    // Holes can help clarify what is happening.
    val handleState = handler {
        x => \s. (x, s);
        incr(v, k) => \s. k(())(s + 1)
    };

```

```

with handleState handle {
  compute(1) + compute(2)
}

// Please try to insert holes in the above code
// where you think it is helpful, and try to understand
// how handleState is used, how the state is updated
// and how the final result is produced.
}

/*
+-----+
| Task5_2: State      |
+-----+
*/

// Try to implement the operation get and put.
// 'get()' returns the current state and 'put(s)'
// updates the current state to the given argument s:
{
  with handler {
    x => \ s . (x, s) ;
    get(x, k) => \s . k(?) (?) ;
    put(v, k) => \s . k(?) (?)
  } handle {
    val x = #put(42);
    #get(())
  }
  (1)

// -->* (42, 42)
}

/*
+-----+
| Task6: Nondeterminism |
+-----+
*/

{
  // Nondeterminism can also be expressed by effect handlers:

  // The result of 'choose()' is non-deterministic, and it
  // decides the result of this computation.
  // We can specify the behavior of 'choose()' by defining a handler:

  // Task1: define a handler that collects the max result
  // of the non-deterministic computation:
  val max = \x y. if x > y then x else y;

  val chooseMax = handler {
    x => ?;
    choose(x, k) => ?
  };

  //Task2: define a handler that collects all the possible results

```

```
// of the non-deterministic computation, into a list:

// concatenate lists
def concat(xs)(ys) = xs match {
  [] => ys;
  x :: xs => x :: concat(xs)(ys)
};

val collectAll = handler {
  x => x :: [];
  choose(x, k) => ?
};

with collectAll handle
  if #choose(()) then 1 else 2
}
```